

Autonomous Driving with Computer Vision and AI

By Albert Caballero Crespi



Treball de Recerca

2n de Batxillerat

Escola Mare de Déu de la Salut

Tutor: Sergi Bonet

16/10/2023

Abstract

The rapid advancements in autonomous driving and artificial intelligence (AI) have brought a new era of transportation, providing better safety, comfort, and efficiency. The project explores both fields and analyses the basis for autonomous driving systems based on computer vision. The aim of this project is to develop a system that is capable of performing the basic tasks involved in autonomous driving, such as lane keeping, obstacle detection and pathfinding. Through the use of simulations and a physical prototype, the system's performance will be rigorously tested, showing a possible real-world development and application of such a system. The project will provide a better understanding of autonomous driving and AI, while also showing a practical insight into the practical development of such cutting-edge technologies.

Acknowledgements

This project would not have been possible without the help of a few people. First of all, I would like to thank my tutor, Sergi Bonet, which, firstly, believed in the outcome of this project, and then he has guided me throughout its entirety.

Secondly, I would like to thank the entire team at the “Institut d’Investigació en Intel·ligència Artificial”, who provided computational resources that enabled me to train the object detection model. Especially, I would like to thank Pedro Meseguer, who put me in contact with the IT department and made the petition to the director of the IIIA.

I would also like to thank the UPC’s EcoRacing Team, especially Pau Climent, which aided me early on in the project, providing me with documentation and resources and helping me focus down on primary objectives.

Last, but not least, I would like to thank all of my family and friends, which have been extremely supportive throughout the whole project.

Index

1. Introduction	4
2. Objectives	5
3. Autonomous driving	6
3.1 Advanced driving-assistance systems (ADAS)	6
3.1.1 Cruise Control	7
3.1.2 Adaptive Cruise Control	7
3.1.3 Lane Keeping Assist/Lane Centring	8
3.1.4 Conditional Driving Automation	8
3.2 Existent autonomous cars	8
3.2.1 Tesla FSD	8
3.2.2 Waymo	9
4. Artificial Intelligence (AI)	11
4.1 Defining and evaluating intelligence	11
4.2 Defining Artificial Intelligence	12
4.2.1 Neural networks	12
4.2.2 Classification by goals	13
4.3 Evolution of Artificial Intelligence in the past	14
4.3.1 The Enigma Machine	14
4.3.2 Start of Artificial Intelligence	14
4.3.3 Deep Blue	15
4.4 Artificial Intelligence in the present and its applications	15
4.4.1 Generative AI	15
4.4.2 Commercial applications	16
4.5 Ethical issues	17
4.5.1 The trolley problem	17
4.5.2 Data collection and privacy	17
4.5.3 Bias	18
4.5.4 Usage rights and ownership	18
4.5.5 Deepfakes and false information	18
5. Prospective usage	19
6. Technological design	20
6.1 Software	20
6.1.1 Lane detection	20
6.1.2 Obstacle and traffic sign detection	20
6.1.3 Pathfinding and navigation	21
6.2 Simulations	22
6.3 Physical prototype and hardware	22
6.3.1 Physical prototype	22
6.3.2 Hardware	22
7. Creating the line following system	24
7.1 Defining the approach	24

7.1.1	Lane detection with AI.....	24
7.1.2	Image Processing approach.....	24
7.2	Detecting the lanes.....	24
7.2.1	Image processing.....	25
7.3	Vehicle control outputs.....	28
7.3.1	Determining the centre and the error	28
7.3.2	Steering the vehicle.....	29
8.	Creating the object detection AI.....	30
8.1	Testing pre-trained object detection models.....	30
8.1.1	YOLOv8	30
8.1.2	YOLOv8-seg	31
8.1.3	Faster R-CNN.....	32
8.1.4	YOLOP	32
8.2	Training an object detection model	33
8.2.1	Making the dataset.....	33
8.2.2	First attempt – YOLOv8 w/ GTSDDB	34
8.2.3	Second attempt – YOLOv8-seg w/ COCO traffic	35
9.	Pathfinding	37
9.1	Defining the approach.....	37
9.2	Pathfinding algorithms.....	37
9.2.1	Dijkstra.....	37
9.2.2	A* (Astar)	38
9.3	Real-world case example	39
10.	Implementation in simulations.....	40
10.1	Interacting with the game.....	40
10.1.1	Getting images from the game.....	40
10.1.2	Providing feedback back to the game	40
10.2	Code infrastructure	40
10.3	Testing the lane following system	41
11.	Physical implementation.....	44
11.1	Prototype details	44
11.2	Code adaptations	44
11.3	Found issues	45
11.4	Testing the line detection system.....	45
12.	Technical details	47
12.1	Line Following system.....	47
12.2	Object Detection system	47
12.2.1	Testing an object detection model	48
12.2.2	Training the object detection model	49
12.3	Downloading and installing	50
13.	Conclusions	51
14.	References.....	52

1.Introduction

The realm of autonomous driving has had an increase in importance in the past years. From increased safety to comfort, many are the advantages that it offers. The recent developments on the field of artificial intelligence (AI), and its involvement in computer vision have allowed for significant advances on how images can be perceived by computers, providing powerful tools for autonomous driving. With these improvements, many companies are starting to promote such technologies, which have shown to be powerful in other applications apart from its usage in the automotive industry.

Both the field of AI and automotive innovation are of strong personal interest. This project has served as a hands-on experience to develop fully fleshed out software, while going through the process of iteration and overcoming difficulties, which are present in real situations.

The aim of this project is to serve as a proof of concept of how these technologies work and are developed. After having a thorough insight into the realms of autonomous driving, and AI, more specifically computer vision; the main objective will be to develop a system which is capable of performing the basic tasks required, for autonomous driving, which are lane keeping, obstacle detection and pathfinding. This will require the use of various AI-based techniques.

To validate the performance of the driving system, it will be implemented into simulations and a physical prototype. This will be able to actively provide feedback throughout the entire development of the system and also approximate how this system would work in a real environment.

The structure of the project will be as follows: Firstly, the different approaches to autonomous driving and the current state of AI will be analysed. Afterwards, the system, which will include the most significant features required for autonomous driving, will be developed. Finally, it will be tested in simulations and with the physical prototype.

2.Objectives

Creating an autonomous driving system

The main aim of this research project is to develop an autonomous driving system. Because making a fully capable system involves a lot of hardware that is not too accessible and developing software that is out of scope, it will not be possible to make a fully capable self-driving system. In other words, the autonomous driving system that will be created will be limited in performance and accuracy and will not be capable of being implemented in a real car because of its limitations.

In short, the objective of the project will be to create and implement some of the functionality of self-driving/ADAS systems involved in it, serving as proof of concept.

Testing and training the driving system in simulations

Testing will play a fundamental role in the development of the autonomous driving system. Apart from using it to improve the system, it will be necessary for the development, as some of the technology will involve AI (mainly for computer vision) and, consequently, simulators will be needed to train the system.

This will be done by developing a relatively simple simulation (the limitations and functionality of which will be specified later) that will allow to recreate an environment that is close enough to reality to train the driving system and test its functionality. Apart from simulations, images and sets of data will be used to train the AI for computer vision (there will be more details about training and AI later).

To summarize, for the development and training of the driving system, simulations will have to be used to recreate a real-life alike environment.

Testing the driving system in real life

The final aim of this project is to test it in a real-life environment. Based on what can be achieved there are two options to do so:

Eco Racing: an Eco Racing car will be used to test the driving systems. These vehicles were already designed with the aim of autonomous driving for racing and most of the hardware needed for it is already installed. With the help of the Eco Racing team at UPC, the software can be implemented into their car and then put to the test in a circuit.

Model car: If using the Eco Racing car is not a viable option, a model car, and a model environment such as a test track can be used to test the driving system. To make this environment, all the aspects and challenges that the system should be able to overcome will be considered. Hence, it will be possible to evaluate and validate the system more precisely.

3. Autonomous driving

Throughout the past years, there has been an increasingly pursued aim in the automobile industry of providing aidance to driver. Currently, almost every car has some sort of system that helps the driver in some way. From cruise control to full self-driving systems, the technologies are varied and have evolved vastly in the past years.

3.1 Advanced driving-assistance systems (ADAS)

An advanced driving-assistance system (ADAS) is any piece of electronic hardware or technology that assists the driver when driving and/or parking, with the objective of making the vehicle safer and improving the driver's performance. ADAS can be both passive and active systems.

Passive ADASs, such as TCS, ABS, Blind Spot Detection, and Parking Assistance Systems, are designed to only provide information to the driver, and rely on it taking action to avoid an accident. Active ADASs, such as Automatic Emergency Breaking and Adaptive Cruise Control, can make actions when required. The combination of these systems is the basis of autonomous driving.

Depending on the level of autonomy/functionality of these systems they fall under 6 levels:

Level 0: Only information is provided to the driver and the ADAS does not have any control over the vehicle.

Examples: Parking sensors, Blind Spot Detection.

Level 1: The ADAS can take control over only one functionality to assist the driver.

Examples: Adaptive Cruise Control, Emergency Break Assist.

Level 2: The ADAS assists in acceleration/breaking and steering. It still requires the full attention of the driver.

Examples: Lane Keeping and Adaptive Cruise Control (simultaneously) Autonomous Parking.

Level 3: Also known as conditional autonomous driving, the ADAS allows the vehicle to make decisions based on traffic and other considerations. These often involve the use of AI or more advanced processing.

Level 4: The vehicle can autonomously make decisions and fully control the car. It does not require a driver to take over driving. It can only be if all required conditions are met. The vehicle does not require pedals and a steering wheel.

Level 5: The vehicle can autonomously drive under any circumstance.

(The 6 Levels of Autonomy Explained, 2022)

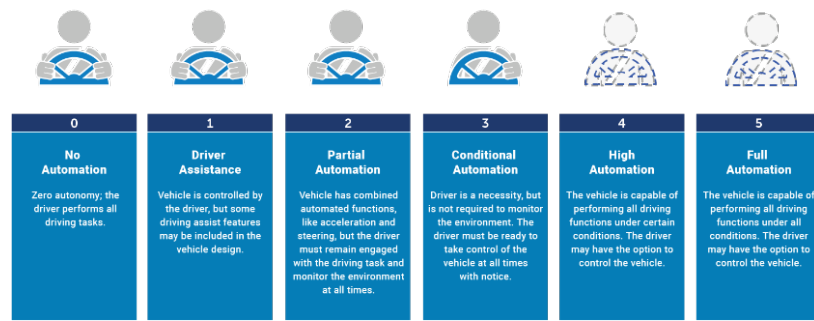


Figure 1. Levels of autonomy (Source: Accolade Technology)

A few examples of ADAS systems are the following:

3.1.1 Cruise Control

Cruise control (also known as speed control) is a level 1 ADAS system that can keep the vehicle at a certain speed set by the driver. It works by adjusting the engine's throttle to maintain the speed. This system was invented in 1948, but new improved systems have been developed to replace it. The main limitation of this system is that the car has no way to know what is ahead of it and the driver must be constantly changing the speed. (Kobylinski, 2019)

3.1.2 Adaptive Cruise Control

It is a more advanced version of cruise control is adaptive cruise control, which was first implemented in cars in the 90s. Apart from maintaining the vehicle's speed, this system can also break and accelerate automatically to keep the distance to the car ahead, giving it the ability to sense near traffic.

There are different types of adaptive cruise control:

- Radar detection (the most used)
- LiDAR
- Sonar
- Camera-based

The main difference between these types is the sensor used to get information about the environment and how it reacts to it. (Kobylinski, 2019)

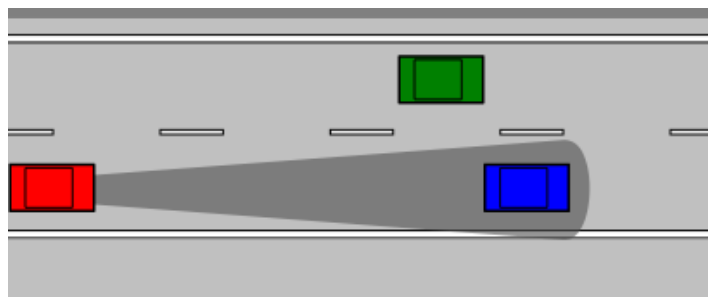


Figure 2. Representation of adaptive cruise control (Source: Wikimedia Commons)

3.1.3 Lane Keeping Assist/Lane Centring

This system actively steers the vehicle to keep the car centred within the lane. It does this by detecting the lines of the lane either with cameras or radar and LiDAR-based sensors.

Apart from helping the driver when driving on long, monotonous journeys by relieving the driver's necessity to steer, it also acts as a significant safety measure, as it prevents the vehicle from drifting out of this lane even when not engaged.

This ADAS plays a big role in autonomous driving as, together with adaptive cruise control, it makes for the basic functionalities for controlling a car which are accelerating/breaking and steering.

Adaptive Cruise Control and Lane Keeping separately are considered level 1 ADAS and level 2 when used in combination with the two. (Laukkonen, 2021)

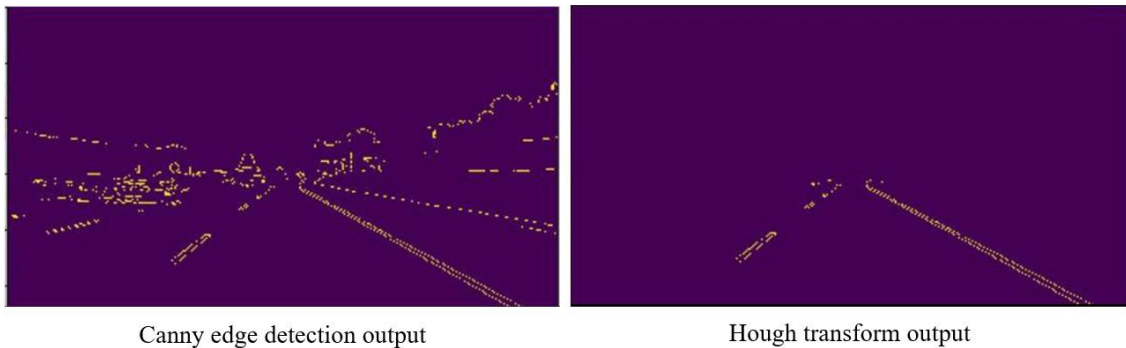


Figure 3. Camera-based Lane detection (Source: Wikimedia Commons)

3.1.4 Conditional Driving Automation

The difference between level 2 autonomy and level 3 can appear to be subtle at first, it is a substantial improvement from a technological standpoint. Level 3 systems give the vehicle the ability to analyse the environment and other vehicles to make decisions autonomously. To achieve this more advanced sensors and processing are required. (The 6 Levels of Autonomy Explained, 2022)

3.2 Existent autonomous cars

As of October 2023, there currently exist no production cars with level 3 systems or above, as the conditions required for this system are very specific and the technology is too expensive and requires exhaustive development. However, cars are being developed which will have this functionality. (Boora, 2023)

3.2.1 Tesla FSD

One of the most popular autonomous driving systems is Tesla Full Self-Driving, which is still in development. Currently, the technology has the capability of a Level 3 system, but due to legal and testing reasons, the driver is still required to pay full attention and must be prepared to intervene at any point, therefore making it a Level 2 system.

How it works

For FSD, Tesla vehicles use 8 cameras and 12 sensors to gather information about the environment and surroundings. By scanning and modelling its environment based on this information, and with the use of AI, it plans a path for the car to follow that takes into consideration factors such as obstacles and prioritizing other drivers and pedestrians.

The AI has been trained with driving footage and data collected from Tesla cars. Throughout the years of development, it has been able to constantly improve, getting closer to a driving model that is closer or even better than human driving, and potentially safer.

Impact and safety

Tesla's technology has proved to improve safety significantly. According to Tesla's Vehicle Safety Report (Tesla, 2023), in 2021 drivers who were using Autopilot technology had an average of 4.85 million miles driven per crash, and Tesla drivers who were not using these technologies had 1.4 million miles per crash, while the average in the US was 652,000 miles. This shows a very notable increase in safety in comparison to other vehicles.



Figure 4. Tesla with FSD engaged (Source: Wikimedia Commons)

3.2.2 Waymo

Waymo is an autonomous driving technology company that is known for its fully autonomous cars. Based on production cars, their cars are modified to have level 3 ADAS capabilities.

Waymo started in 2009, taking up the challenge by Google to drive 160 kilometres autonomously without interruption. Through changes in the vehicles used, they kept improving until in 2017 they released the first public demo of their autonomous taxis. In 2020 they made their services available in Phoenix, Arizona and San Francisco, California in the US. Nowadays they are also investing in new projects related to logistics and autonomous driving. (Waymo, n.d.)

How it works

Waymo's autonomous driving vehicles use a combination of sensors and technologies. It uses a combination of LiDAR, cameras, radar, microphones (to hear sirens from far away), GPS, and other sensors. With these sensors, it maps and scans its environment, and it predicts and anticipates the actions of other vehicles, pedestrians, and other dangers. (How Autonomous Vehicles Work, 2021)

Impact and safety

After one million miles of autonomous driving, Waymo's fleet has not had any reported injuries from accidents. According to Waymo's report on the Safety Performance of the Waymo Rider-Only Automated Driving System at One Million Miles (Waymo, 2023) their cars had only had 20 incidents, of which 9 did not result in any damage and 40% were parking-related. From these events, only 2 are CISS-comparable (meaning that they would have been reported to the police in a normal situation, as minor car crashes are often not reported). Comparing this to CISS (Crash Investigation Sample System) data, it is shown that Waymo's autonomous driving can improve vehicle safety, being less likely to be involved in an accident than the average US vehicle.

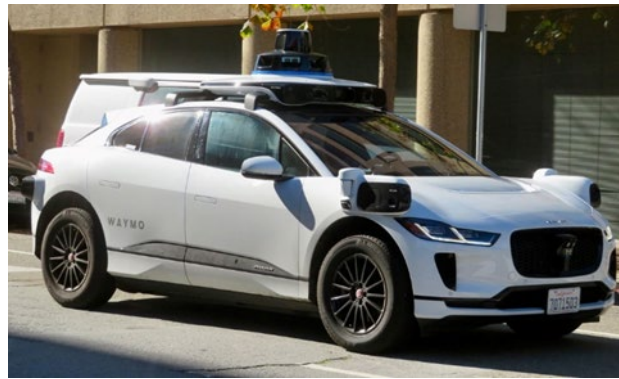


Figure 5. Image of a Waymo car (Source: Flickr)

4. Artificial Intelligence (AI)

From machines that can beat the world's top players at chess like Deep Blue, to being able to answer questions and have a conversation with humans like ChatGPT; AI is and will become prevalent in almost every aspect of our daily lives.

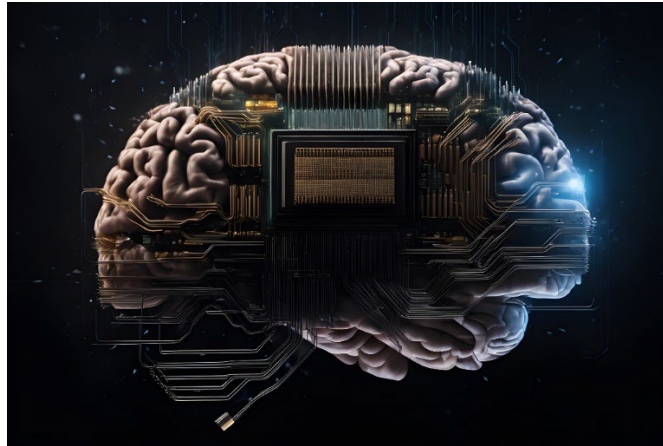


Figure 6. Representation of artificial intelligence made by Playground AI using Stable Diffusion XL (own content)

4.1 Defining and evaluating intelligence

Artificial intelligence is the field of creating machines which show intelligence. However, it is difficult to determine whether a machine is “intelligent”, as it is difficult to define intelligence itself, especially when applying the term to a machine. Throughout history there have been different approaches to determining if a machine is intelligent:

In 1950, Alan Turing defined intelligence as a machine's ability to show intelligent behaviour. On the other hand, John McCarthy (known as one of the fathers of AI) says intelligence is the ability to achieve goals. In general terms, what makes an AI intelligent depends on its ability to perform tasks, adapt, learn, reason, and solve problems effectively, often in a manner that resembles human intelligence. (McCarthy, 2007)

To test a machine's intelligence, Turing created the Turing test. The Turing test, also known as the Imitation game, consists of a human and a machine designed to generate human-like responses in which a human evaluator tries to tell which one of the two is a machine. The machine passes the test if the evaluator is not able to distinguish reliably which one of the two is the machine. To this date, no computer has been able to pass the Turing test, but some, such as ELIZA (Weizenbaum, 1966), have been able to come close to doing so by managing to fool some people. (Johnson, 2022)

The Turing test, however, has been heavily criticised. Firstly, the Turing test is limited to recognizing the ability to engage in human conversations and is incapable of evaluating actions. Apart from this, another issue it has is that the intelligence of AI should not be determined by comparing it to humans, as intelligence is not limited to human-like behaviour. As Stuart J. Russell and Peter Norvig said, “Aeronautical engineering texts do not define the goal of their field as making machines that fly so exactly like pigeons that they can fool other pigeons.” (Russell & Norvig, 2020).

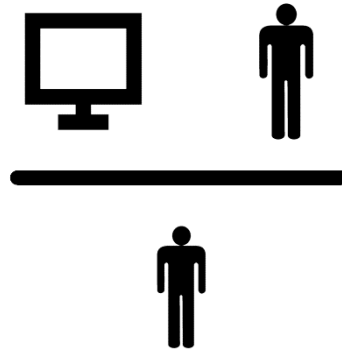


Figure 7. Representation of the Turing Test (own content)

Throughout the years, variations of the Turing test have been developed and are very relevant nowadays, such as the Reverse Turing Test, in which a human tries to convince a machine that it indeed is human. An example of the Reverse Turing Test is CAPTCHA. (Alizadeh, Mniestri, & Stevens, 2022).

4.2 Defining Artificial Intelligence

Despite AI being a commonly used term, few people know the true technical definition and aspects that make an AI.

4.2.1 Neural networks

In its essence, an AI is a neural network. Similar to how human brains work, a neural network is an array of layers made of nodes which are connected with the nodes of the following layer. Each connection between the nodes has a weight.

The first layer consists of the input nodes. The input nodes represent the given variables. Depending on the objective of the neural network, the number of input nodes can vary. In an image, there will be a node for each colour value (R, G, B) of every pixel of the image. For example, for to process a 640x640 image there should be a total of $640 \times 640 \times 3 = 1.228.800$ nodes.

The last layer, known as the output layer, has a node for each one of the possible answers the neural network gives. In a dog breed classification model, for example, there will be a node for each possible dog breed. When processing the images, each node will have a value of how certain it is that is the correct answer.

Between the input and the output layers, there are the hidden layers, which add extra complexity to the neural network. Its nodes are connected with the nodes of the previous hidden layer (or the input layer) and the nodes of the next hidden layer (or the output layer). The difference between Machine Learning (ML) and Deep Learning (DL), which are the most common types of AI algorithms, revolves around the number of hidden layers. The more hidden layers there are, the more precise the model will be. However, it will require further training and processing power. For example, ChatGPT in total has 12288 layers and 175 billion (thousand million) parameters (weight values).

The weights that connect each one of the nodes are firstly assigned at random. The goal of training an AI is to find the values these weights should take to accomplish the objectives. The most common way to get the desired weights is by using a “back propagation” algorithm. Once the neural network processes some data, and the mistakes at the outputs are known, the back propagation algorithm works its way back from the output layer modifying the previously random weights in order to reduce the error.

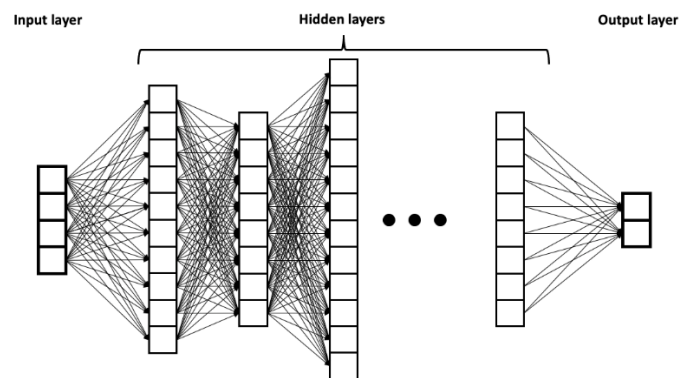


Figure 8. Representation of a neural network (source: Wikimedia Commons)

4.2.2 Classification by goals

AIs can be classified in many ways. By their objectives there are various types, the most notable of which are the following:

Logical computing

These are limited to computing the best answers on a logical environment where there is no hidden information. These are common in games, such as sudoku solvers and chess programs (such as Deep Blue).

Natural Language Processors

Natural Language Processors (NLPs) are dedicated to understanding the human language and generating human text. They are present in chatbots, translating software, etc...

Perception

These AIs are designed to understand its environment, either from images, lidar or with other sensors. There most important for autonomous driving, the subject of the project.

4.3 Evolution of Artificial Intelligence in the past

Since the appearance of computers, there has been an exhaustive search for intelligent computers. In 1950, Alan Turing wrote a paper discussing the possibility of using computers to solve problems. However, at the time computers were expensive to run and unable to accomplish such tasks.

4.3.1 The Enigma Machine

The first major instance of usage of exhaustive computing power was in 1940, amid World War II, with the objective to decode the Enigma Machine used by the German army to encrypt their communications. Alan Turing and the team at the UK Government Code and Cypher School designed the Bombe, which deciphered enigma-encrypted messages by selectively trying all the possible combinations.

In 1940, the allies were able to successfully decrypt the German communications. This gave the allies a major advantage, as they were informed of all attacks and operations. Without computers, the Enigma Machine would have impossible to solve, as it had approximately 151 trillion possible combinations. (Prasad & Kumari, 2020)



Figure 9. Enigma Machine at Bletchley Park (Source: Wikimedia Commons)

4.3.2 Start of Artificial Intelligence

The event that started the field of AI was the Dartmouth Summer Research Project on Artificial Intelligence taken place in 1956, where many researchers gathered to discuss various subjects related to AI. In that event, John McCarthy gave the field the name “Artificial Intelligence”.

Over the next years, computers became more powerful and accessible, allowing for the first AI demonstrations to be made, for example, ELIZA, a natural language program (Weizenbaum, 1966). Due to the rising popularity and success of the field, many AI research programs were funded, and the subject was further developed to the extent that in 1970 it was thought that machines that matched human intelligence would be developed in the coming years. Still, those requirements were unmatched by any computer and research was halted for almost a decade.

In the 1980s new algorithms and techniques such as Deep Learning appeared, allowing machines to learn for the first time. In the 1990s and 2000s began to shine and many objectives were starting to be accomplished. (Anyoha, 2017)

4.3.3 Deep Blue

One breakthrough was the first machine to win a human in the tremendously complex game of chess. The machine in question is Deep Blue, which was able to win against Gary Kasparov in 1997. Previously, they had two matches in 1989 and 1996, which both resulted in Kasparov's win. Still, Deep Blue improved every time and was even able to win one of the six games in 1996. (Goodman, 1997)

4.4 Artificial Intelligence in the present and its applications

In the past decade the improvements and evolution of AI has been massive compared to the past. Computers have since become cheaper and significantly more powerful, consequently enhancing the capabilities of AI programs and making them much more accessible. It has gone to the extent that today anyone can run a Deep Blue level chess bot or decrypt Enigma messages with our phones. Especially in the recent years, the field of AI has suffered a significant growth in development and improvements.

4.4.1 Generative AI

In the past year, certain natural language generators (also known as chatbots) were released to the public which took the world by storm, making society and global leaders rethink the significance and the implications of AI. The most notable of them is ChatGPT, which has shown the capabilities of such technologies, and the effects AIs will have in the future.

The implications of ChatGPT have been unprecedented, and now it is far more accessible and present in aspects of our daily lives than it would have ever been imagined. ChatGPT and other chatbots can generate all kinds of texts, in almost any language and about any desired topic.

*In the land where the grill did roar,
A long past war, oh what a lore,
Hot dogs and hamburgers, side by side,
In a battle of buns, with ketchup as their guide.*

*Flames were blazing, in the summer's heat,
As they clashed upon that sizzling street,
Hot dogs sizzled, with a fiery bark,
Hamburgers flipped, in the sizzling dark.*

*Buns stood tall, like castle walls,
As mustard cannons fired in the brawl,
Ketchup rivers, flowing crimson red,
On the battlefield where heroes bled.*

*But in the end, they found a way,
To grill in peace, come what may,
Hot dogs and hamburgers, we'll unite,
On the picnic table, 'neath the starry night.*

Poem generated by ChatGPT 3.5. Prompt: "Write a poem with rhymes which talks about a long past war between hot dogs and hamburgers." (Source: own content)

Though chatbots can be really useful in our daily lives (for example explaining topics that would be hard to comprehend), their main potential revolves around their ability to accomplish tasks that would otherwise require mechanical, repetitive work done by humans. Text translation and transcription, for example, will soon be overtaken by AI, as AI-based alternatives are much more efficient and vastly cheaper to maintain than humans.

Apart from that, not only are AIs capable of generating outputs that closely resemble human behaviour, but, as it can be seen in the poem above, they can also perform tasks that would normally require creativity. This quality is not limited to text, though, as there are generative AIs with the ability to create images and videos, recreate voice and make music, and even making 3d models, just by giving them a line of text describing what the output should be like.



Figures 10-11-12. Images generated using Perplexity AI, based on Stable Diffusion XL (Source: own content)

Generative AIs, however, are still flawed in many aspects, as they do not really “understand” the output they are giving, they are limited to generating media which they predict that correlates to the input prompt. This results in consistency issues and other kinds of mistakes. In other words, despite looking correctly, the media generated by AIs can be “wrong”. In the first image, for example, the raccoons are supposed to be on the moon and, still, the moon can be seen in the background. And in the second image, there are issues with the perspective, as the legs of the teddy bears are in front of the bars in the back of the shopping carts.

4.4.2 Commercial applications

From education to manufacturing, many are the advantages offered by AI. In schools, AIs can be used to monitor the behaviour, results, and other aspects of students more effectively, allowing for a more efficient formation of the students that would otherwise require more work by teachers which would be unreasonable when dealing with tens of students.

AI is an excellent tool for the management of great amounts of data which, together with the amount of data that can be retrieved from social media and cookies, has been one of the most appraised aspects by businesses.

In medicine, AI has also had a rise in popularity in the past years. A lot of research has shown that its usage, especially the usage of object detection models, has a lot of potential in some areas, for example the diagnosis of melanoma (a type of skin cancer) from images with the use of object detection models. These have proven very accurate, and even much more accurate than dermatologists when only visually analysing them, without the aid of any equipment. (Ahmadi Mehr, 2022)

Moreover, robots have made an appearance in some hospitals. They can substitute doctors and nurses, doing simpler activities such as interacting with a patient, which were very effective during the pandemic; and they can also transport samples or equipment, carefully navigating the environment in order to avoid any obstacles.

4.5 Ethical issues

With the increase in popularity of AI, many ethical concerns have been raised about its usage and other aspects.

4.5.1 The trolley problem

The trolley problem is a famous dilemma which was first discussed by the British philosopher Philippa Foot. In its imaginary situation, there is a train speeding towards five people that are on the train tracks. However, there is a lever that can be pulled which would deviate the train to another track with only one person in it. The dilemma revolves around whether it is correct to make an action to save five people but sacrificing one person which would have otherwise been uninvolved.

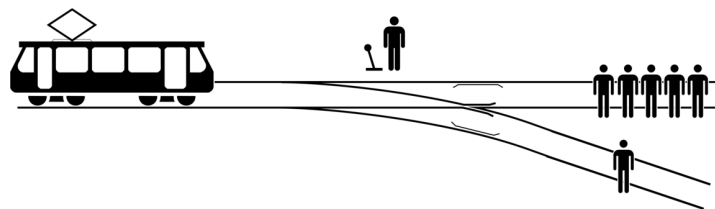


Figure 13. Representation of the trolley problem. (source: <https://snl.no/sporvognsproblemet>)

There are many variations of this dilemma. In the realm of autonomous driving, it is of paramount importance, as a similar situation may occur in which an autonomous driver has to decide to either sacrifice the passengers, or the pedestrians (or other similar situations).

4.5.2 Data collection and privacy

It is known that in many places, such as China, AI face recognition software is being used to track its citizens. This provides the governments with all kinds of information which can be processed with other programs, giving them the option to pursue the citizens which perform actions against the government's regime.

4.5.3 Bias

Another issue with the usage of AI, especially with chatbots and other generative AIs, is its tendency to promote the ideologies from the data they have been trained with. ChatGPT for example, was trained on data acquired from Reddit, a well-known online forum. On the first versions of the chatbot, it showed slight tendency to a racist and misogynist behaviour, which was common on the online forums. Even though the information used to train can be filtered in order to not have bias, it is impossible to have a completely neutral ideology in the dataset.

4.5.4 Usage rights and ownership

With the appearance of image generation AIs, many lawsuits were launched against the companies that developed them. The object of the lawsuits was that these companies did not own the rights of the images which were used to train the models. To this day it is still unsure, despite the images being generated by the AIs, if the created images are owned by the creator of the prompt, the companies behind the software, or the millions of owners whose images were used to train them.

4.5.5 Deepfakes and false information

A few years ago, videos made with AIs started to appear online. They showed famous people saying and doing controversial things which resulted being fake. For example, in 2018 a video was released in which the former president of the US Barack Obama, talked about the dangers of videos which could show false information. The person talking was in reality the actor and comedian Jordan Peele which, with the use of AI, was able to recreate the voice and images of Obama.

Sooner or later, the images generated with AI will be undistinguishable from real images. This can result in the spreading of false information, and it is argued that the media generated by AI should always have a “watermark” clearly differentiating them from real media.



Figure 14. Deepfake of the pope wearing weird clothing (source: unknown)

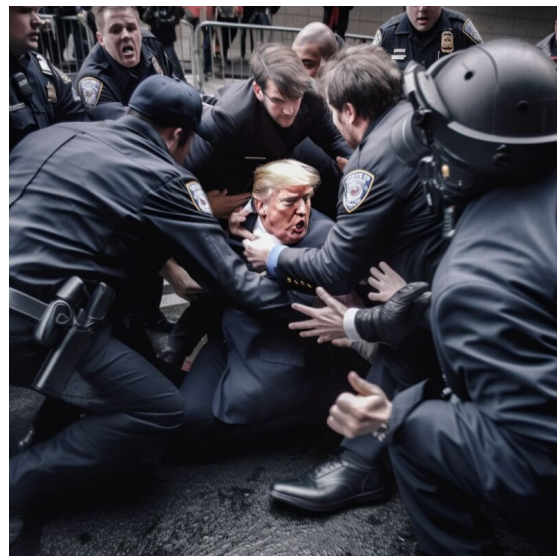


Figure 15. Deepfake of Donald Trump resisting to being detained (source: unknown)

5. Prospective usage

Taking the information about autonomous driving and AI discussed above into consideration, it is clear that autonomous driving is a challenge difficult to tackle, which is not possible to completely overcome with the constraints of this project. Not only does it involve a lot of resources and time to develop a good autonomous driving system, but it also requires a lot of experimentation and iterating. The final driving system will have some limitations.

Firstly, complex sensors will not be used, as all the systems will be based around cameras. Cameras are cheap compared to other sensors, like LiDAR, though imagery requires more intensive processing, requiring the use of computer vision and/or AI.

Secondly, the system will not have all the functionalities required for a fully capable autonomous driving system. Making autonomous driving systems has requirements that are simply out of reach. The driving system will be kept simple, and it will only have the most basic features needed for autonomous driving. It will still be able to drive on its own, but it will sometimes be faulty, slow, imprecise, and it will not be able to handle some situations and obstacles.

With the consideration of these limitations, the functionality of the final product will be the following:

- Lane detection and lane keeping: this will allow the vehicle to detect the boundaries of the road and keep it within them.
- Object/obstacle detection: the system will be able to detect obstacles and objects and react to them accordingly, either by avoiding them or breaking. Another example use of object detection that will be implemented is traffic sign detection.
- Pathfinding and navigation: an algorithm that can make a route to get from point A to point B which then navigates the vehicle.

As for the simulators, if used, they will also be kept as simple as possible. Because the driving system will be based on vision, it will be necessary to simulate a real-life environment with visuals and physics. Creating a realistic visual representation of reality is a hard task on its own and it requires a powerful computer to simulate, but it cannot be the other way either, as by making the visuals too simple, it will not be able to recreate the real environment and the prototype will not perform well enough. In the end, this will be a trade-off between the needed resources and the realism of the simulation.

Having developed the autonomous driving system, it will have to be tested out. As previously said, it will be done by using said simulations and finally, it will be used in a physical prototype, such as a car from Eco Racing (if possible) or a model car. It is worth mentioning that, to implement the driving system into any physical prototype, some modifications and limitations will have to be made to adapt it to the prototype's abilities and hardware. In this sense, the modifications made for this will require having to rewrite some parts of the code, as the firmware and the software for the control of the vehicle were specifically designed for it.

To summarize, the final deliverables of this project will be the software for the autonomous driving system, the simulation framework to test and train it, and a model car to test it in an environment closer to reality. It is also worth mentioning that both testing tools could be used for other driving systems.

6. Technological design

As mentioned before, this project will require the development of three main things: the driving system (software), the physical prototype and hardware, and the simulations. These do not have to be developed in a specific order although some parts of the development require other parts being made, for example, to make the software, simulators may be needed. Thus, they will be developed in parallel.

6.1 Software

Creating a driving system requires a complex process, which can vary depending on its objectives and what technology it is using.

In this case, to simplify things, the driving system can be split up into separate systems (lane detection, obstacle detection and pathfinding/navigation), which then they can be combined with a control system, which gets the outputs of the other systems and then communicates with the hardware (steering actuators and motors) to steer and drive the vehicle accordingly.

6.1.1 Lane detection

This system revolves around computer vision (CV) and will be able to distinguish the lines in the lane from the rest of the images. There are many techniques that can be used to achieve this, the most common of which being CV and AI, the latter of which requiring training.

The different approaches to lane detection will have to be analysed. To test them and achieve their best performance by tweaking the parameters and configuration, image and video footage of a dashboard camera (which will be recorded personally or taken from online datasets) will be used.

The main approach will be warping, filtering, and distinguishing the lanes from the image with the use of colour-filtering, and analysing the pixels of the image. This will not necessarily require the use of deep learning, or any other AI based algorithm if it succeeds. But until the performance of this system is good enough, tuning and/or training will keep going. If the results do not improve, this approach will be discarded and another one will be used.

6.1.2 Obstacle and traffic sign detection

The system required for obstacle and traffic sign detection is similar to lane detection, as it is also based around CV, but getting lane detection to work is much simpler and requires less training or no training at all.

This system, along with lane detection, is on its essence image recognition, and there many open-source algorithms designed for this. The benefits of these algorithms are that they have already been designed and fine-tuned to be used in roads for the same purposes as in this case, but they usually require more computing power and they have added functionalities that will not be used in this case. An example of this is YOLO (Joseph Redmon, 2016), which is an open-source object detection system designed for real-time object detection.

The object detection system will be tested in the same way as the lane detection. Apart from its performance and accuracy in detecting the objects, the performance (as in time it takes to process images) will also be considered as a significant factor, as it will determine how fast the system can react to a situation.

Based on what camera(s) and sensors are used, the approach for the two previous systems may be different. For example, using stereo cameras makes it possible to determine the distance of objects while a combination of normal cameras can be used to achieve the same effect.

6.1.3 Pathfinding and navigation

This will rely on the previously mentioned systems, as the path the vehicle will follow will be determined by the lane and the obstacles it must avoid. It will also be divided into multiple systems:

Route Planning

This will be responsible for taking the vehicle from point A to B, such as in GPS navigators. The algorithm will have to calculate one of the most optimal routes based on the roads that connect the two points. From this route, it will create waypoints for the car to follow. This will require either an algorithm or an AI model.

Navigation

Having the waypoints for the vehicle to follow, it will be necessary to write an algorithm that connects those waypoints while avoiding any obstacles. There also are many approaches to this. To find the one that performs the best, they will have to be tested out in simulations.

Once those systems are developed and tuned, and they perform accordingly, they will have to be combined through a control system which will make the decisions for the vehicle to follow. It will take the inputs from the various systems and control what the hardware should do. This serves as a kind of control system.

This system will also keep track of the position of the vehicle, either with GPS, accelerometers, and other sensors; or by calculating the position of the vehicle based on the motor's power and the steering; and it will implement safety measures such as emergency breaking.

6.2 Simulations

Throughout the development of the driving system, the use of simulators will make aspects of the testing and training of the systems easier. There already are open-source simulators such as CARLA, developed by the Centre de Visió per Computador (CVC) (Koltun, 2017). They require a lot of graphical computing power, a constraint in this case; so, making own simulators may be a better solution. It will be developed either on Unity or Unreal Engine and it will consist of two main parts: the graphical design, which will simulate a photorealistic environment; and the physics of the car.

6.3 Physical prototype and hardware

6.3.1 Physical prototype

The physical prototype requires the design of the structure as the prototype, the “chassis”. The design will be made considering the sensors and actuators it will need, the mechanism to steer and other features it will involve.

It will be designed using CAD or any 3D modelling software. After that, it will be built. The materials and how it will be done is yet to be determined.

6.3.2 Hardware

With the prototype structure and design done, the hardware will have to be developed. This includes all the sensors, actuators, chips, processors, and other components. The hardware used for the prototype will be made according to the final requirements and properties of the driving system.

Actuators

It includes the motors and the servos for the steering. They will be chosen depending on the size of the prototype and what speeds it is expected to achieve.

Sensors

The type and amount will vary the most depending on the driving system. Apart from the cameras and, as previously said, the vehicle will also require other sensors to be able to perform correctly. These will mainly be GPS sensors and accelerometers (to track the position of the vehicle), and ultrasound sensors, which will be used either to calibrate the cameras or as safety measure for emergency breaking.

Chips and processors

The processors and chips used will also be chosen depending on the system. The main factor which will determine what processor is used will be the performance requirements for the driving system. A possible candidate for a processor is a raspberry pi or any other equivalent alternative.

The components used for the prototype will also be chosen with the objective of trying to keep the costs the lowest possible while also having a good performance. After choosing them, they will be mounted onto the chassis and then they will be connected accordingly.

After designing every physical aspect of the prototype, the structure will be manufactured, the components will be sourced and then it will all be assembled. Finally, the software will be uploaded to the chip and adapted to proceed with the testing phase.

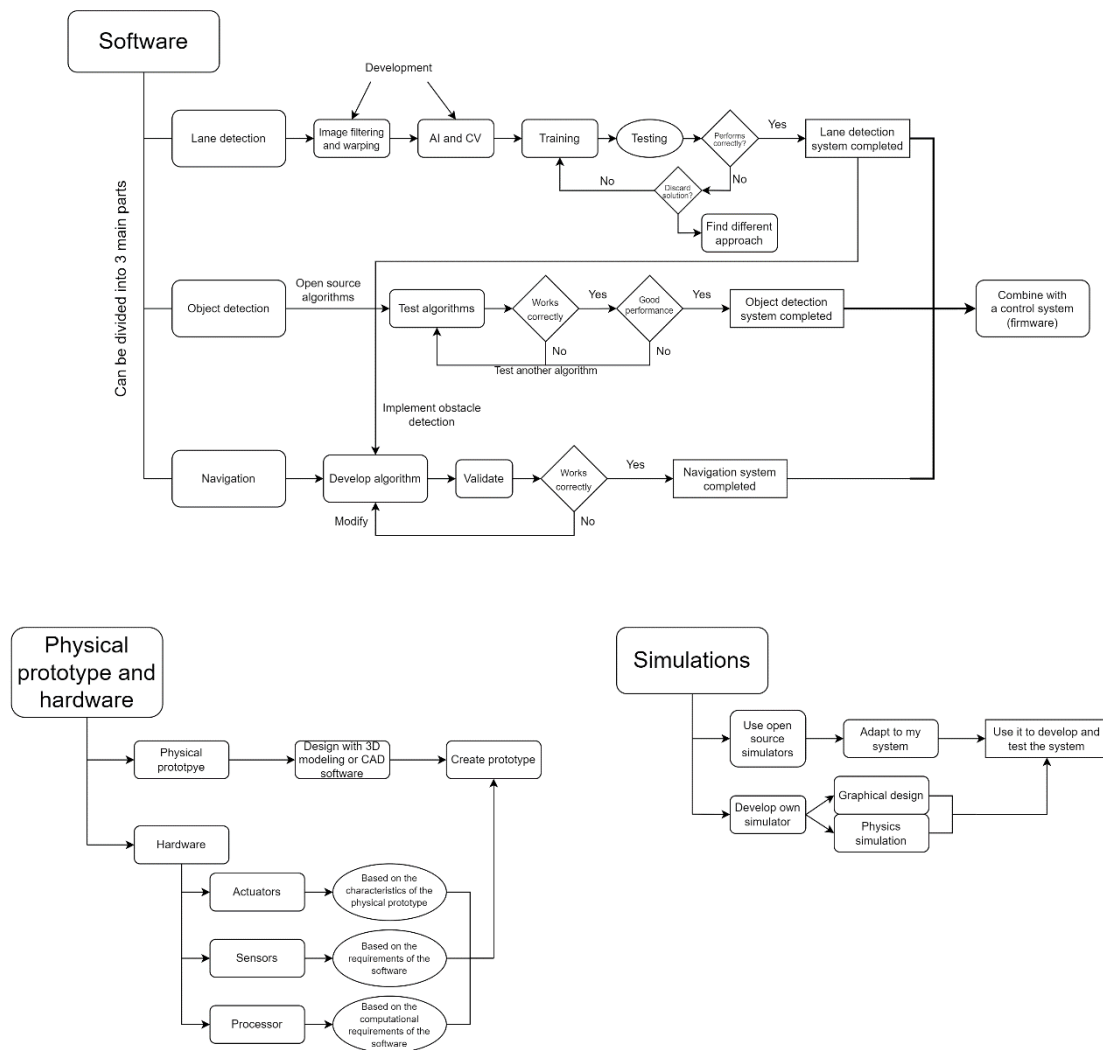


Figure 16. Diagram of the technical design (source: own content)

7. Creating the line following system

7.1 Defining the approach

There are diverse approaches to making a line following system. The main two taken into account for the project are analysing the image with the use of AI and processing the image (without the use of AI), a more “mathematical” approach.

7.1.1 Lane detection with AI

The approach around AI consists of training an AI model to detect the lanes, then extract the information from the detected lanes (slope, intercept/displacement) and predicting the direction that the vehicle should take. The main drawbacks of this approach are the substantially higher computational requirements, and the necessity to have an extensive dataset with accurate lane information.

There are some AI models that perform these tasks, such as YOLO Panoptic (see 8.1.4). However, extracting and interpreting the information outputted by these models can be difficult, as they are more focused on “painting” or masking the detected lanes on top of the image.

The advantages that AI models offer is the ability to implement multiple functionalities which also involve the use of AI at the same time. For example, the model could be able to both detect the lanes and the other objects (cars, people, and other), merging the multiple functionalities into one system.

7.1.2 Image Processing approach

The other approach, which is significantly simpler and requires much less processing power, relies on directly analysing and interpreting the pixel information from the image (brightness, colour...) to detect the lanes.

The latter will be the one used for the project, as it directly processing the image is much less complex and easier to implement in simulations or physical prototypes than by training and using AI models.

7.2 Detecting the lanes

To keep the car within the lanes, first the system must be able to detect the lanes and determine the car’s position relative to the lane lines. There are many approaches to detecting the lanes, such as detecting the lane markings themselves or identifying the drivable surface.

7.2.1 Image processing

To be able to detect lanes with the use of cameras it will be needed to process the image. The images will be processed in real time, in order to get constant information about the lanes. The images in question will look like the following:

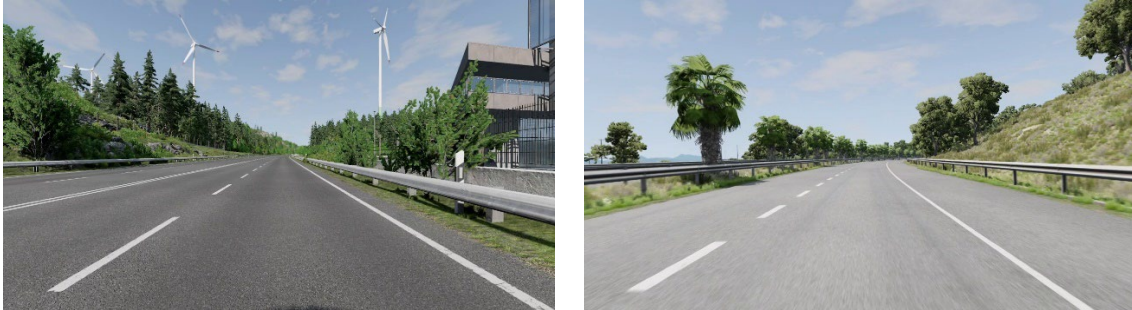


Figure 17-18: Screenshots taken from Beam NG (own content)

Acquiring valuable data directly from these images, however, is a difficult task, as there is too much unneeded information and noise, so it needs to be filtered.

Firstly, colour can be removed. Normally, in an image the value of a pixel $x, y = (R, G, B)$. However, as lanes are white, the colour values can be done away with. To do so, the image can be converted into grayscale.



Figures 19-20: Grayscale images of original screenshots (own content)

By converting the image into grayscale, the value of a pixel is simplified to the intensity of the pixel (or brightness). As a result, the new value of a pixel $x, y = I$. Now it is possible to extract the lanes by analysing brightness of the pixels of the image. By looking at the intensity values of the pixels of a row the next graph is observed:

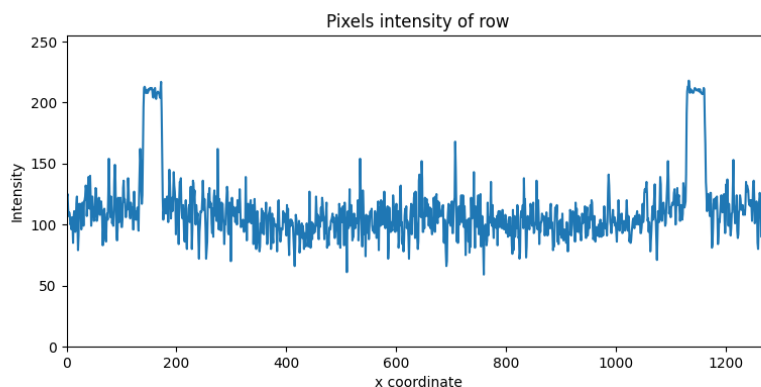


Figure 21: Graph of pixel intensity values (own content)

In the graph, sudden intensity changes can be observed. These indicate lanes, as the pixels change from dark (the pavement) to bright (the lane marking). However, there still is too much noise in the image. To fix this, a gaussian filter can be applied.

The gaussian filter works by averaging the value of a pixel with other neighbouring pixels. This effectively blurs the image and, as a result, the noise. The result given when applying the gaussian filter to the image and getting the intensity values is the following:

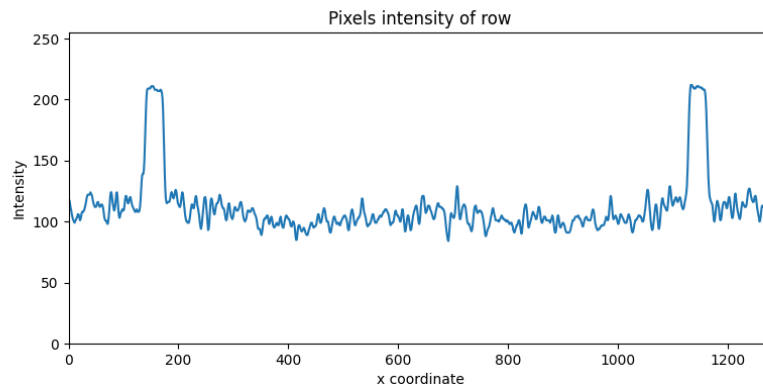
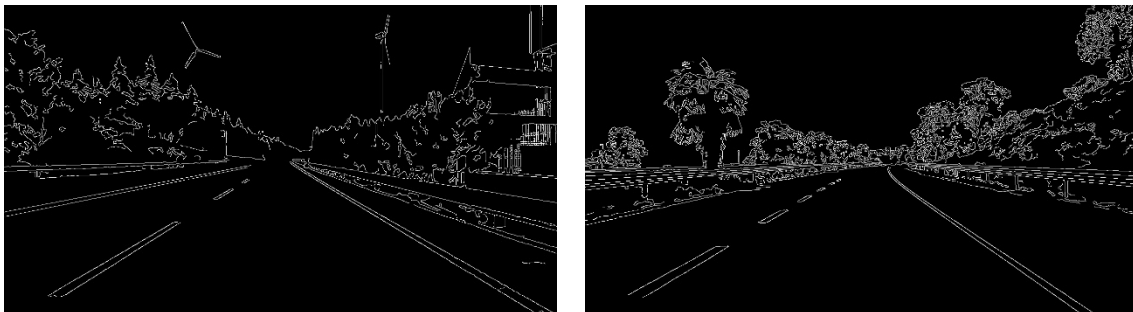


Figure 22: Graph of pixel intensity values of gaussian blur image (own content)

Now that the image has been simplified and the noise reduced, an algorithm can be applied to detect the intensity changes mentioned above. In this case, the algorithm “Canny Edge Detection” will be used.

When running the Canny Edge Detection algorithm through the blurred image, the result is the following:



Figures 23-24: Canny Edge Detection results of blurred grayscale images (own content)

The lanes can be easily detected now. However, there still are many features in the images that need to be removed, mainly the surrounding trees, and the elements in the background. To filter out those elements, a mask can be used.

The mask has the same shape as the image. Within the mask a shape is drawn, giving each pixel a value of either 1 or 0. With the mask, each pixel of the image is multiplied by its equivalent in the mask, thus turning the parts of the image that is not included in the mask, as they are multiplied by 0. In this case, the mask covers the area where the lanes are expected to be located. The shape of the mask will depend on where the camera is pointing at and its field of view. The mask is the following:

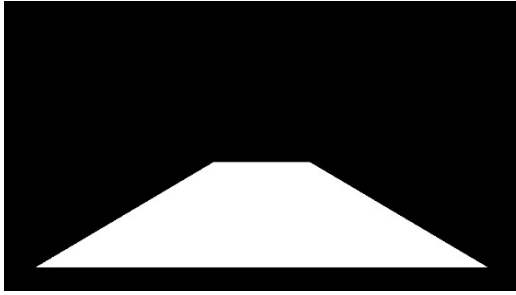


Figure 25: Mask image (own content)

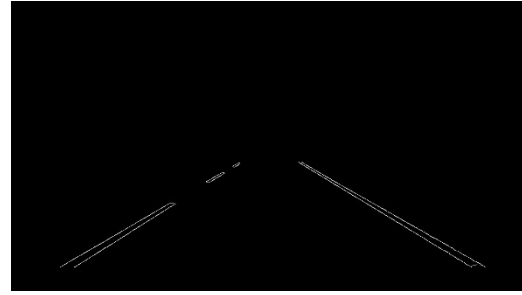


Figure 26: Masked canny edge image (own content)

Now the lines are the only elements present in the images. However, the pixel data only represent points, and it needs to be converted lines. To do so, there are a few algorithms that transform points into lines. In this case, the Hough Lines Transform will be used, which outputs $(x1, y1), (x2, y2)$ for every line that is detected:

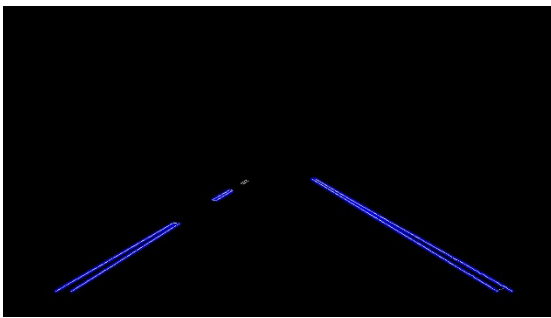


Figure 27: Detected lines drawn on top of canny (own content)

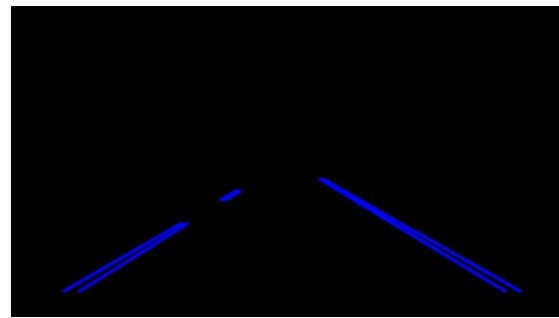


Figure 28: Drawn detected lines (own content)

The code is now able to detect the lanes from an image. The script can output a list of all the lines it detects. The list of lines looks like this:

```
lines = [[ 794, 449, 1144, 660], [722, 400, 939, 525], [123, 659, 395, 502], [263, 594, 408, 503], ...]
```

Despite successfully detecting the lines, too many are detected, as we only need one line for the left lane marking and another for the right one, the lines from the lists have to be classified into the two groups. This can be achieved by checking the slope of each line. Taking the horizontal axis in the image as the y axis, lines can be defined with $y = mx + n$, with “m” being the slope and “n” the intercepts (or displacement). Therefore, if $m > 0$ the line is part of the right lane markings and vice versa. The resulting two lines are the average of the left and right groups.



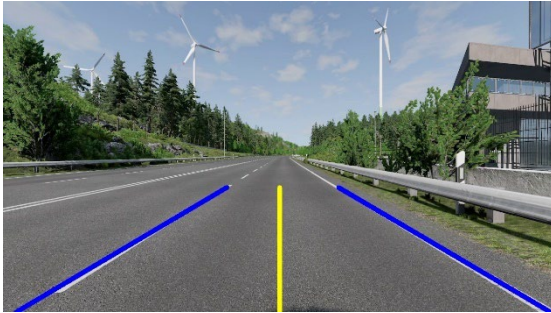
Figures 29-30: Detected lanes drawn on original screenshotted images (own content)

7.3 Vehicle control outputs

Now that the lane lines are detected, the vehicle must be steered to keep it in the centre of the lane. This will determine how the vehicle should steer to correct its deviation.

7.3.1 Determining the centre and the error

Nevertheless, it is firstly needed to determine what the “centre” is. In this case, as the camera is perfectly centred relative to the car direction, therefore (as the horizontal axis in the image has been taken for the y axis) the line defining the centre path of the image will be $y = 0$. However, in a real environment this might not be true, so this may have to be tuned for each case.



Figures 31-32: Images with detected lanes and centre line (own content)

The other line required is the centre of the lane. It can be calculated by averaging the slope and the intercepts of the left and right lines.



Figures 33-34: Images with detected lanes (blue), centre line (yellow) and central lane line (red) (own content)

Now that both the vehicle's and lane's centre lines are defined, it can be known how the vehicle is positioned and oriented relative to the lane and how it needs to steer.

To know how the vehicle's position/orientation should be corrected, an error function is used. This error function takes into account both the slope and intersect difference. Each of the parameters has a weight to establish how strong of an effect each parameter has. The function used for this project is as follows: $\text{error} = k_{\text{intersect}} \cdot \Delta n + k_{\text{slope}} \cdot \Delta m$

7.3.2 Steering the vehicle

Now that the error has been calculated, it can be used to decide how the vehicle needs to steer to return the vehicle to the centre. It cannot be directly used, however, as a control algorithm is required. The control algorithm will make sure the vehicle does not over steer, or, in other words, it does not try to overcorrect the error.

For the sake of simplicity, a PID will be used. The Proportional-Integral-Derivative controller (PID) is a control algorithm used to correct the error between a continuously changing variable and a setpoint. It responds based on the proportional (proportional to the current error), the integral (the cumulative error), and the derivative (how rapidly the error is changing) terms. Each of these terms has its own weight. The mathematical formula of the PID is the following:

$$u(t) = K_p e(t) + k_i \int_0^1 e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

Each of the weights of the PID can be tuned. There are various methods of doing so, like with software or approximating the behaviour of the whole system with mathematical formulas. In this example, however, they have been manually tuned, which in this case is more than enough for the control system to perform properly.

The PID output is then clamped to between -1 and 1 and controls the angle of the wheels.

8. Creating the object detection AI

For the object detection system, vision-based AI models will be used. Firstly, already made models will be tested. After this, new object detection models will be created and trained.

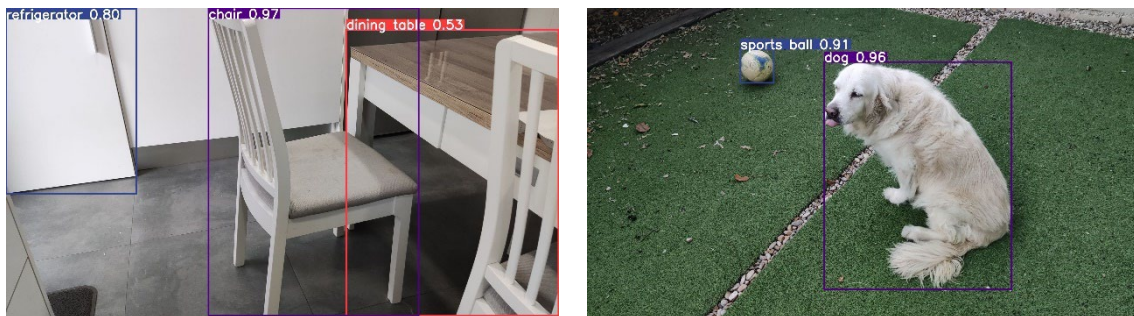
8.1 Testing pre-trained object detection models

Note that these models have been pre-trained on the COCO (Common Objects in Context) dataset. This means that the models will have the ability to detect the objects from this dataset. The COCO dataset is commonly used as a benchmark test for object detection AI models.

8.1.1 YOLOv8

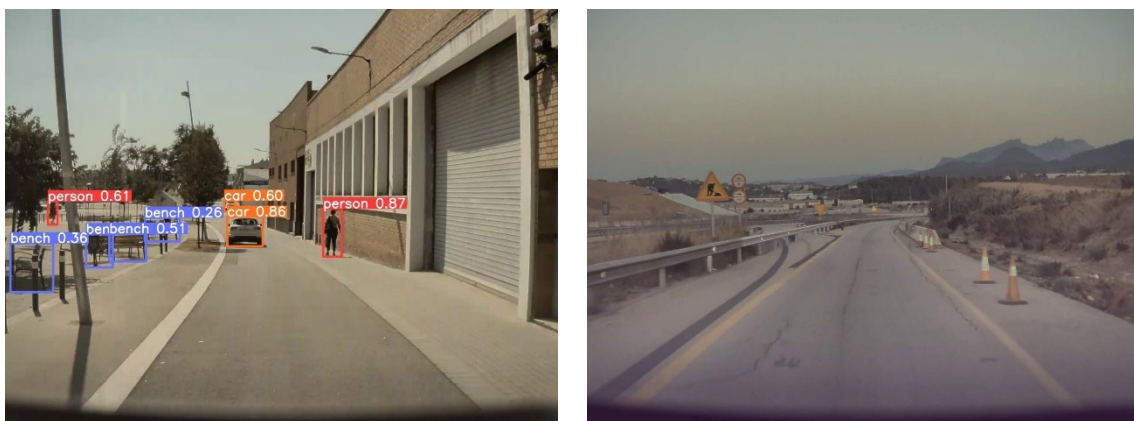
You Only Look Once (YOLO) is a real-time object detection model. YOLO processes an image and outputs the objects it detects, its confidence (how certain it is that the detection is correct), and `x1, y1, x2, y2` points that create a bounding box on the original image where the object has been detected.

There are many versions of YOLO, the latest being YOLOv8. Each version improves performance. Apart from that, each version also has different variations, which have more, or less, accuracy for the sake of computing requirements and processing time.



Figures 35-36. Examples of everyday objects detected with YOLOv8-m. ~300ms to process (own content)

As it can be seen in these images, even though it incorrectly detected a refrigerator, the model was successfully able to detect the objects in these images.



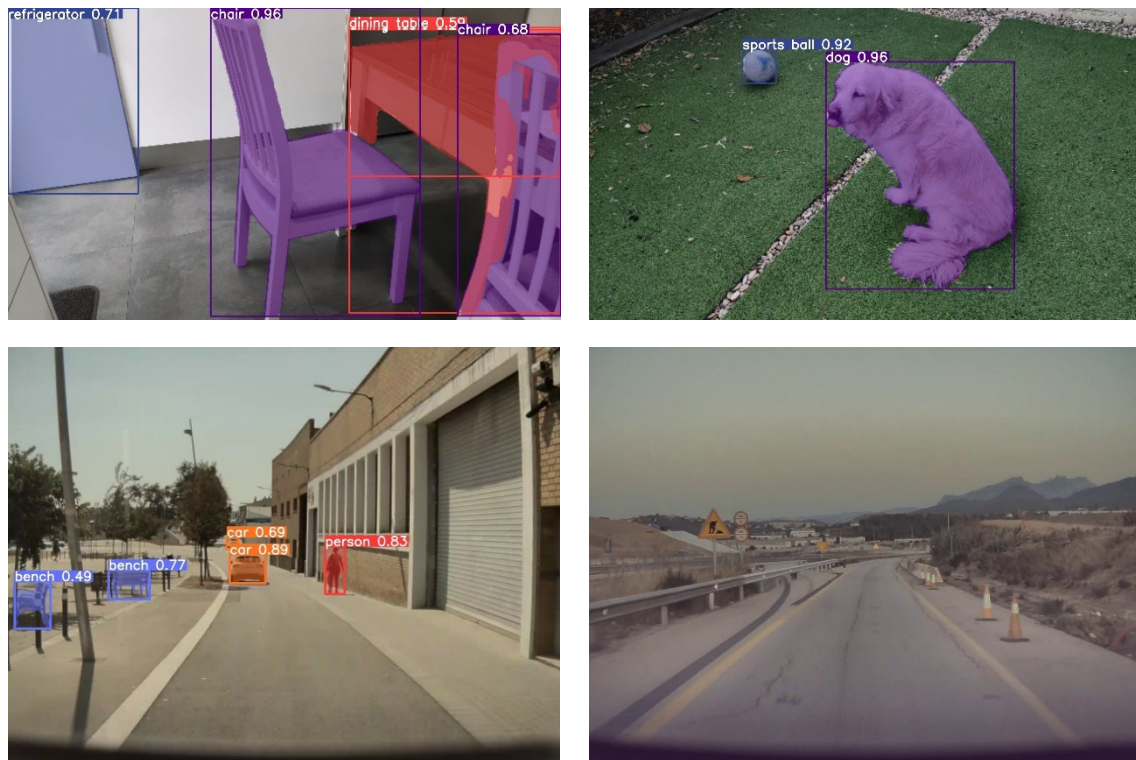
Figures 37-38. Examples of objects detected from road images with YOLOv8-m. 500ms & 470ms to process (own content)

When running the images through the model, its issues become more apparent. This model has been trained on a dataset, called COCO (Common Objects in Context), which is aimed towards detecting everyday “objects”, such as animals, people, furniture, food... (Lin, et al., 2014)

Not all the objects in the dataset are relevant in this case and some others are missing. In the first image, detecting the benches on the side is unnecessary. Even though they could be ignored, it would not be optimal for the performance. In the second image on the other hand, objects such as traffic signs and cones are not detected, which are of great significance.

8.1.2 YOLOv8-seg

This model is also based on YOLO. The main difference it has is that instead of outputting the boxes where it detects objects, it outputs the points for a polygon that overlays such object. This is called “semantic segmentation”, which instead of giving bounding boxes, outputs masks on the objects.



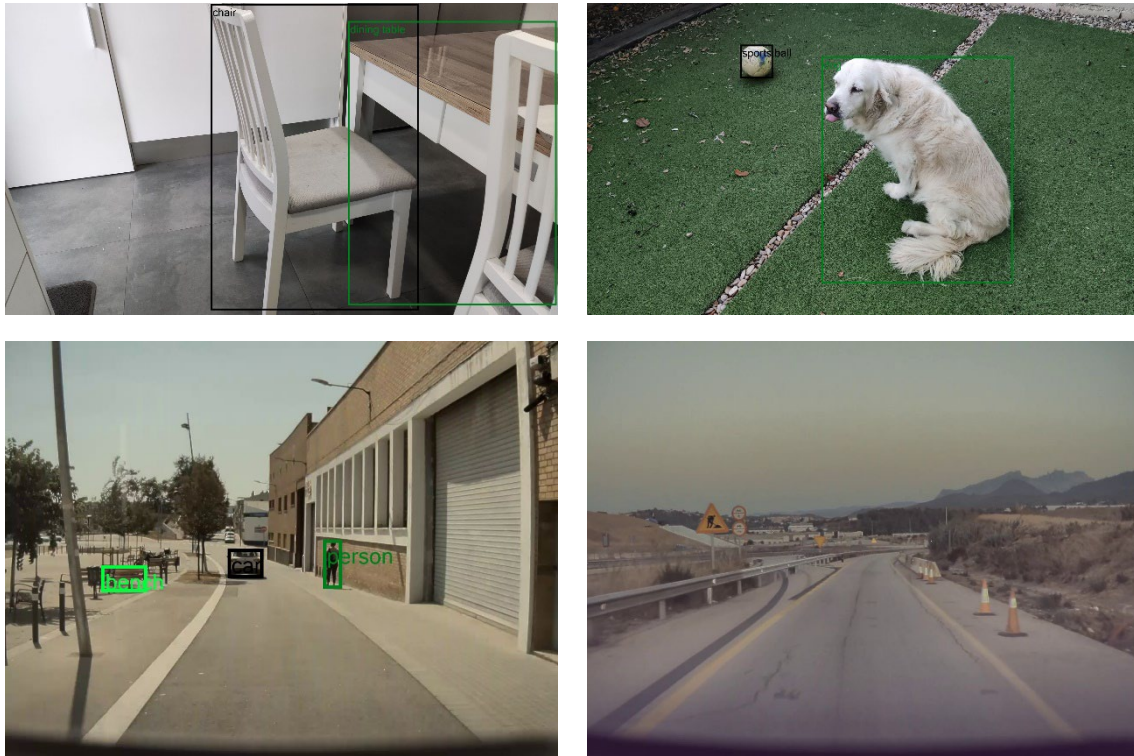
Figures 39-40-41-42. Previous images now ran through YOLOv8-seg-m. 370ms, 720ms, 540ms & 440ms to run (own content)

Even though it takes as much as twice the time to process the images compared to the base model, the advantage of using is that it gives substantially more information about the objects it detects. A case in point where this could be important in the context of autonomous driving, is that it can allow for the estimation of the distance at which an object of known size is, for instance, a sign.

Also bear in mind that the other problems that the base model had are still present.

8.1.3 Faster R-CNN

Faster R-CNN is another type of a convolutional neural network developed by Microsoft, released in 2015. (Ren, He, Girshick, & Sun, 2015). This other object detection model, although much simpler, has proven to be significantly worse in terms of performance compared to YOLO, taking almost 10 times more time to process each image.



Figures 43-44-45-46. Images processed by FasterRCNN. 3809ms, 3660ms, 3261ms & 3206ms to run (own content)

8.1.4 YOLOP

You Only Look Once for Panoptic driving perception (YOLOP) is a variation of the YOLO object detection model, which is able to both identify and segment the drivable surfaces (lanes) lane markings, and detect objects such as cars. (Wu, et al., 2022)



Figures 47-48. Images processed by YOLOP.

This model can perfectly identify the road, lanes, and vehicles; however, it is not able to detect pedestrians.

8.2 Training an object detection model

Training an object detection model consists of using a custom dataset to make a pre-trained model or a new model learn to detect such objects. This gives the model the ability to recognise the desired objects, which are present in the dataset, therefore shaping the model to have the desired capabilities.

More specifically, when training, the model will go through all the images and, with the labels, learn to identify the objects. The training is divided into “epochs”, which are the iterations. After each iteration another set of images will be used to evaluate the performance of the model.

In this case, training a model is necessary, as it will allow it to detect the objects which the default models were not able to, such as traffic cones and signs.

8.2.1 Making the dataset

A dataset, as its name says, is a set of data. In other words, it is a collection of information. In computing/machine learning the dataset is to be manipulated by the computer, and it can contain various types of data such as images, texts, audio, videos, numerical data points, etc. In object detection, more specifically, the datasets are collections of images with annotations. These annotations have been previously made by a human or, in some cases, by a machine; and they have information about aspects of the image.

A dataset for classifying dog breeds, for example, could be like the following:



Figure 49. Golden retriever (Source: “pxhere.com”)



Figure 50. German Sheperd (Source: Wikimedia Commons)

Image 1 = “golden_retriever”

Image 2 = “german_sheperd”

Object detection datasets usually contain hundreds, if not thousands, of images with annotations indicating what objects are present in each image with the coordinates of their respective bounding boxes. In this case, the dataset will have to contain thousands of images with various vehicles, people, and other objects that the model should be able to detect.

There are different options when it comes to making a dataset to train an object detection model which suits the desired needs.

The first option is to create a model from scratch. This would imply taking thousands of images, probably from road footage either filmed personally or taken from open online sources, and then painstakingly annotating all the people, cars, traffic signs, and other relevant objects manually.

Although there is software that aids the process of creating datasets, this would still require a considerable amount of time.

The other option is to find a model that suits this project's needs. Thankfully, autonomous driving is a heavily pursued task, and there are various publicly available datasets designed to train artificial intelligence models for autonomous driving related tasks. However, datasets still need to be slightly adapted for each model, as the required format for annotations and images can change.

In the end, already-made datasets have been used to train the models. The datasets which have been looked into for this project are Waymo's open dataset (Waymo, n.d.), COCO traffic (a variation of the COCO dataset mentioned above), the German Traffic Sign Detection Benchmark (Houben, Stallkamp, Salmen, Schlipsing, & Igel, 2013), the Mapillary Vistas dataset (Neuhold, Ollmann, Rota Bulò, & Kotschieder, 2017) and the KITTI benchmark dataset (Geiger, Lenz, & Urtasun, 2012).

8.2.2 First attempt – YOLOv8 w/ GTSDDB

The intention for this first attempt to train a model was to make an object detection model with the ability to detect and identify traffic signs. For this, the German Traffic Sign Detection Benchmark (GTSDDB) dataset was used. It contains 600 images of 43 types of signs. The new model was based on the pre-trained YOLOv8 model. The training took approximately 6 hours to complete using the central processing unit (cpu) only.



Figures 51-52. Traffic signs detected by the first model attempt. ~60ms to process. (Original image source: GTSDDB)

Even though this first attempt at training an object detection model was able to detect some of the signs in the images, it has a lot of room for improvement. First of all, it cannot detect all the signs in the image which it is supposed to, such as the “no entry” and the “keep right” signs (figure 52), which were present in the dataset. Secondly, it has very low accuracy when it comes to classifying the traffic signs, in this case only correctly classifying the priority road sign in figure 51.



Figure 53. No overtaking sign processed by this model. (Source: Wikimedia commons) 95ms to process



Figure 54. Dashcam image processed by this model. 87ms to process (own content)

The other issue it becomes apparent when running other images (for example figure 54) which, although have very clearly visible signs which it should detect, it does not detect them due to slight distortions in the image or colour shifting. In figure 53 the problem is that the sign is too big and close, and because the model expects them to be small and far, it is unable to detect it.

Having observed these results, conclusions can be drawn about the problems of this model and dataset.

The main issue with the dataset is its lack of diversity in terms of lightning and sign positioning/distance. This makes images very similar to each other, not enabling the model to precisely be able to distinguish between them. It has a relatively small size, as it only consists of 600 images.

The other main factor that frustrates the model's capabilities is that, in order to process images, they have to be compressed down to a significantly smaller resolution (only 640x640) which results in a loss of information which is especially important when dealing with objects of a small size or far distance. A solution which could solve this problem would be first detecting the signs in the image and, if they are detected, upscaling the image and processing them individually with another model designed to classify signs.

Despite all of that, it is important to point out that the processing time of the images were surprisingly low. In other words, the model turned out to be very fast.

8.2.3 Second attempt – YOLOv8-seg w/ COCO traffic

For this second attempt, the objective was to train a semantic segmentation model with the ability to detect pedestrians, cars, bicycles, motorcycles, trucks, buses, and traffic lights. To do so, the COCO traffic dataset and the base pre-trained YOLOv8-seg were used.

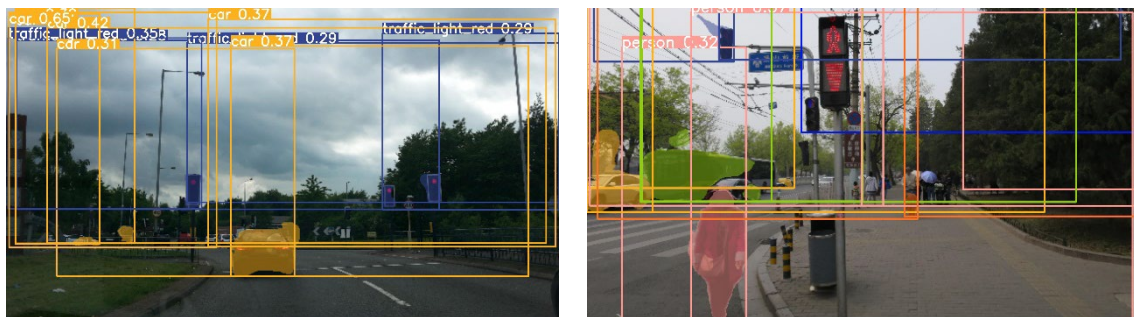
The COCO traffic dataset is a variation of the COCO (Lin, et al., 2014) dataset. It is made of a selection of images from the COCO dataset, only including those which contain the objects mentioned above.

The training required much more computing power than the previous attempt. To address this, and with the help of the Institut d'Investigació en Intel·ligència Artificial (IIIA), it was possible to use the computing resources at the IIIA in order to reduce the training time substantially.



Figures 55-56. Images from the KITTI dataset processed with the trained segmentation model. ~650ms to process

As it can be seen in the images, despite the bounding boxes being off, the segmentation of the objects in the images is quite accurate. The vehicles are correctly detected and marked. In terms of processing time, even though it is higher than the non-segmentation object detection model, it is similar to the base YOLOv8-seg segmentation model.



Figures 57-58. Images from the Mapillary Vistas dataset processed with trained segmentation model. ~1s to process

By looking at these two other images, it can be observed that it is also able to detect more difficult objects such as pedestrians and traffic lights. However, it can be noticed that it is much harder for the model to be as accurate with such smaller objects as it is with cars.

9.Pathfinding

For this project, pathfinding has been looked at from a more theoretical standpoint.

9.1 Defining the approach

In its essence, pathfinding is finding the most optimal route to connect various points. Take the following simplified example: A map is made up of nodes that represent all of the intersections of roads, which are connected by lines that each have a value of the length (or the time it takes to navigate that section of the road).

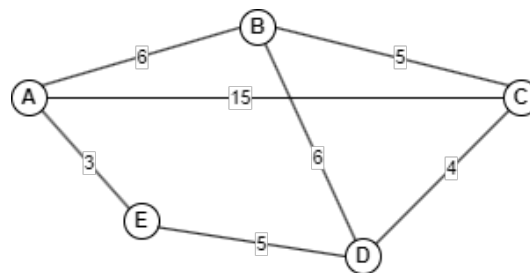


Figure 59. Node map. The circles are the nodes, and the lines are their possible connections with the distance value. (own content)

Considering that the objective is getting from point A to point C, a way to figure out the optimal route is by trying all the possible combinations.

```
routes = [(AB, BC), (AB, BD, DC), (AC), (AE, ED, DC) ...]
```

Then all of the values (distance) for each combination could be calculated and the most optimal route would be found: `optimal_route = AB+BC`

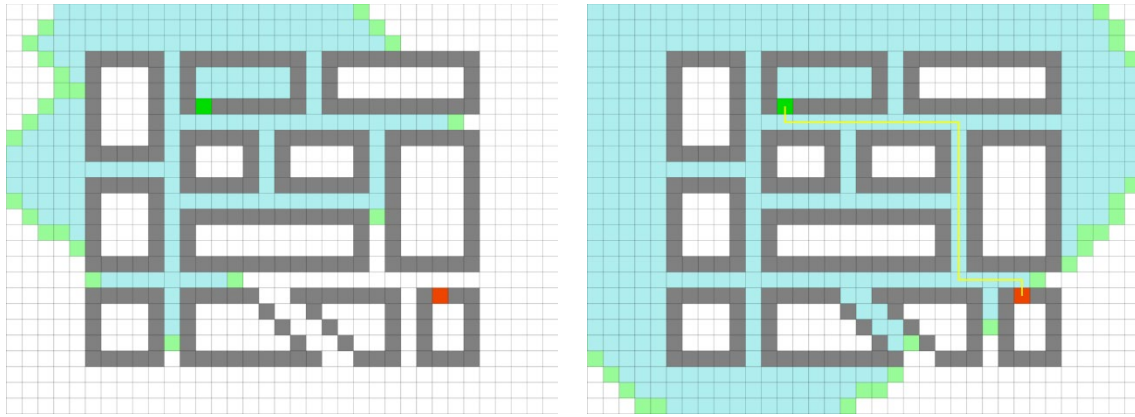
This approach has several issues. In this simple example, it works well enough. However, with a map of an entire city, or even the entire world, for example, the number of combinations that you have to try are impossible to compute. In such cases, many of the combinations do not hold any value. Taking into account every node in the world would make it impossible to compute by testing every one of the combinations. For example, if you want to go from your home to a store in your city, the option of getting to a point in another country does not need to be considered.

9.2 Pathfinding algorithms

A workaround to this is to make use of algorithms. There are many algorithms, each one of which having advantages and disadvantages over others. The most used algorithms are the following:

9.2.1 Dijkstra

The Dijkstra algorithm was created in 1956 by Edsger Wybe Dijkstra. It works by exploring the combination of all the adjacent nodes to the start, then the adjacent nodes of the node which had the shortest distance, and so on.



Figures 60-61. Visualisation of the process of the Dijkstra algorithm. Made using “Pathfinding.js”

As it can be seen in the images the algorithm has to explore a lot of unnecessary points (in blue) to find the optimal path. In this example, it made a total of 1330 operations.

9.2.2 A* (Astar)

The A* algorithm is one of the most used algorithms for pathfinding. It is a variation of the Dijkstra algorithm. To choose what path to explore, the A* algorithm uses a heuristic function which takes into account the cost of getting to the new point and the distance (in a straight line) from the new point to the goal.

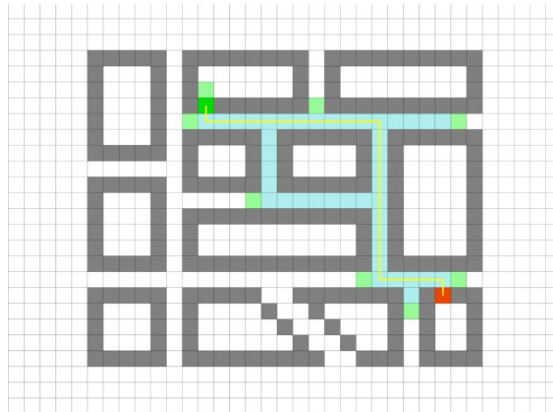
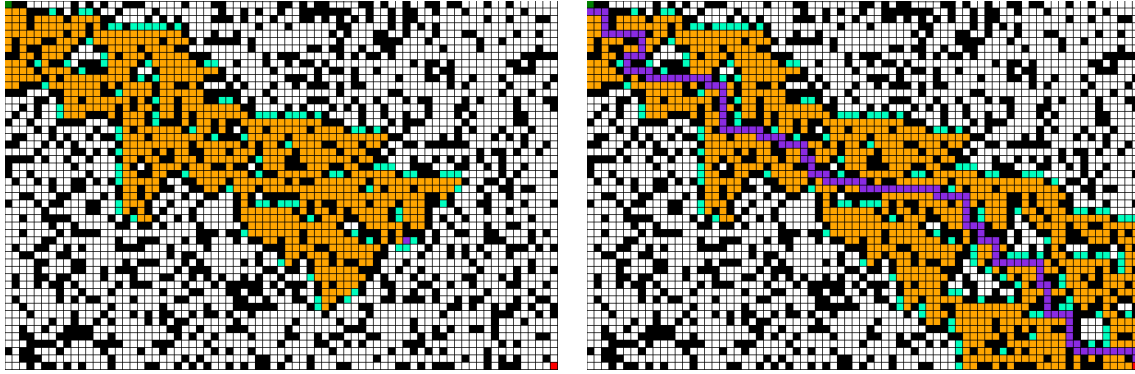


Figure 62. Visualisation of the A* algorithm. Made using “Pathfinding.js” (own content)

As it can be seen in the image, the A* has a significantly better performance, as it does not take into account as many unnecessary points. It required a total of 96 operations, almost 14 times less than Dijkstra.

In these images of a more complex scenario, it can be better understood how the algorithm works. It can be seen that all of the evaluated points are in a generally correct direction, while the Dijkstra algorithm explored the points in a more circular direction:



Figures 63-64. Visualisation of the A* algorithm (own content)

9.3 Real-world case example

While it is not possible in this project to implement a pathfinding algorithm in simulations or in a physical prototype, an example of how these algorithms would work in a real environment will be shown.

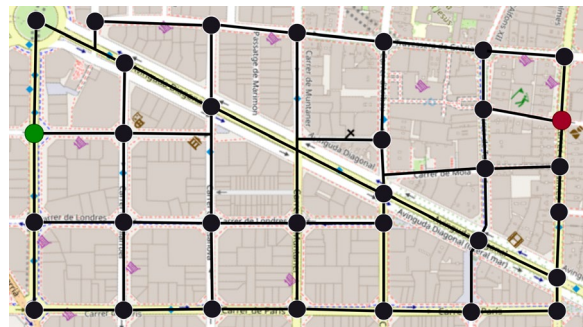


Figure 65. Map with drawn nodes and possible paths (map source: OpenStreetMap)

Take the example case shown in figure 65. It is a map of an area surrounding the “Avinguda Diagonal” in Barcelona. The points represent each intersection between two roads and the lines are the possible paths that can be taken to go from one point to another (note that for this example one-way roads have not been taken into account). Each line would have both a value for length and also the time it takes to cross, as the shortest path is not always the fastest.

With the map, the pathfinding algorithm would start testing the possible combinations (the process would vary depending on the algorithm that is used), taking into account either the length or the time values of each line, until it found the shortest possible path.



Figure 66. Map with some possible paths painted with different colours (map source: OpenStreetMap)



Figure 67. Map with possible optimal route with time values (map source: OpenStreetMap)

10. Implementation in simulations

In the end, due to the complexity of the project, no simulators have been developed. Instead, the driving system has been implemented into the game BeamNG.

BeamNG is a game, launched in 2015, which is most known for its realism both in physics and visuals. It has been chosen for simulating a real environment due to its accurate visuals while being simple enough for the driving system to be implemented.

10.1 Interacting with the game

For the driving system to work in the game, first there must be a way for the game and the code to interact, providing images to the code and then the code providing information to the game to control the vehicle.

10.1.1 Getting images from the game

Neither the game has a feature of providing video output directly to code, nor does the code has a direct way of capturing images from the game. This issue can easily be solved with Open Broadcaster Software (OBS), which is an open-source program commonly used for recording and live streaming. The program captures the screen and then can save the footage or live stream it.

Another feature it has, which is particularly useful to solve this issue, is emulating a web camera. It makes the program behave as a web camera, but instead outputs the footage which is captured in OBS. With this, the code for the driving system can access the footage as if it were a normal camera.

10.1.2 Providing feedback back to the game

BeamNG does not have the option of being controlled with scripts either, however it can detect the inputs from a controller. Thus, an option of controlling the game with code is by emulating a controller in the scripts.

Thankfully, there is a Python library which does this called “VGamepad”. This library emulates DualShock4 (PS4) and XBox360 controllers. The library gives the option to control each button and joystick of the controller, which are then detected by the game as it were a normal controller.

10.2 Code infrastructure

Due to the limitations of common domestic computers (the only available in this case), it is not possible to run the game and the object detection system in real time together. The object detection model which has been trained with the COCO traffic dataset requires a lot of processing power and, unfortunately, the footage from the game will have to be processed afterwards to detect the objects. Despite not being able to react to obstacles or elements in the environment, the system will be able keep itself within the lane.

Its other limitation is that it will not be possible to implement the pathfinding system either, as it is impossible to accurately track the vehicle's position within the game itself without the use of more advanced AI models that map the 3d environment from footage (such SLAM, Simultaneous Localization And Mapping, a type of perception CV model).

Taking this into consideration, the resulting code structure is the following:

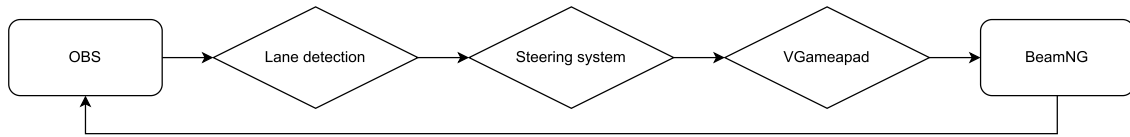


Figure 68. Diagram of the structure of the driving system implementation in BeamNG (own content)

10.3 Testing the lane following system

Overall, the test was a complete success. Not only was the vehicle able to detect and follow the lanes, but it was able to do so without many difficulties at high speeds (120+ km/h). Having said that, even though it performed so well in close-to-optimal situations (good lighting and well-marked lanes without obstructions), even some slight interference, mainly shadows, were able to significantly affect the performance of the lane detection system.



Figure 69. BeamNG capture with a tree shadow (own content)

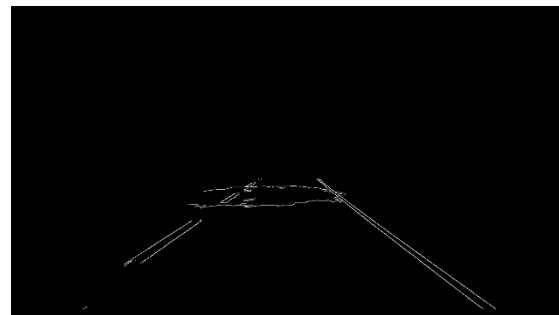


Figure 70. Masked canny edge detection of figure 48 (own content)

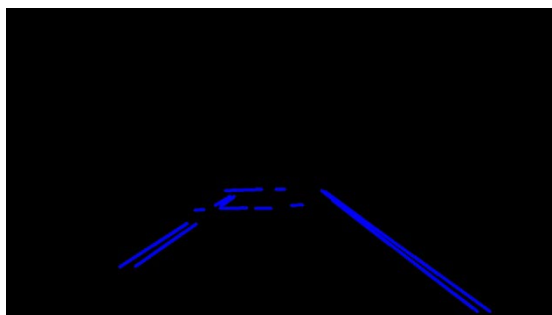


Figure 71. Lines drawn detected lines (own content)



Figure 72. Detected lanes (blue), lane centre line (red) and vehicle orientation (yellow) (own content)

As it can be seen in the images, when there are shadows or artifacts in the image, they may also be detected by the canny edge detection algorithm. This will end up affecting the Hough Lines Transform, adding undesired lines which are not distinguishable from the lane lines, thus altering the centre prediction and possibly steering the vehicle in an incorrect direction.

Other cases that trigger this problem is when there is grass on the sides of the road, which can be still present in the image despite the mask (for example when on a curve), which add a lot of noise to the image even after blurring the image; or when there are signs or arrows drawn on the lane.

In most cases, these two issues are not a problem, as the vehicle is able to correct itself after the shadow or the obstructing objects (such as grass, markings in the lane or other imperfections in the road) are no longer seen shortly afterwards. In other cases where these artifacts are constantly present, such as the road being surrounded by trees, the impact can be of greater consequences.



Figure 73. Intensity graph (figure 14 with proposed thresholds drawn on top).

A solution to this problem could be having a more advanced algorithm to detect the lines (instead of the canny edge detection system) or tuning the canny edge detection algorithm work for when the intensity of the pixels is above a certain threshold.

This however is not useful in all cases, and its effect can vary depending on the situation. This threshold configuration would be very dependent on the illumination. For example, in a darker environment or even when completely in the shadow, the lane markings might be too dark and therefore not over the threshold, resulting in the lanes not being detected.

The other major issue with the lane detection system arises when only one line is either present or it is not visible. Another instance where this is problematic is when the lane widens, and the markings are too far apart, and they are masked off, as if they were an element of the surroundings/background (as it can be seen in figures 74 and 75).

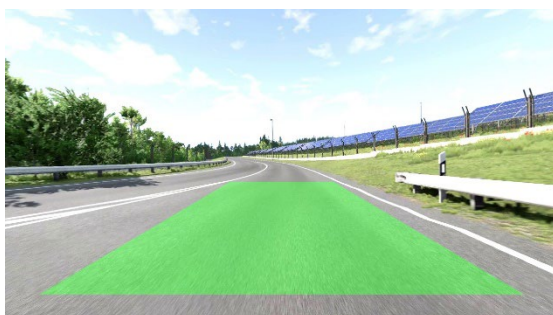


Figure 74. Mask drawn on capture (own content)



Figure 75. Mask drawn on canny (own content)

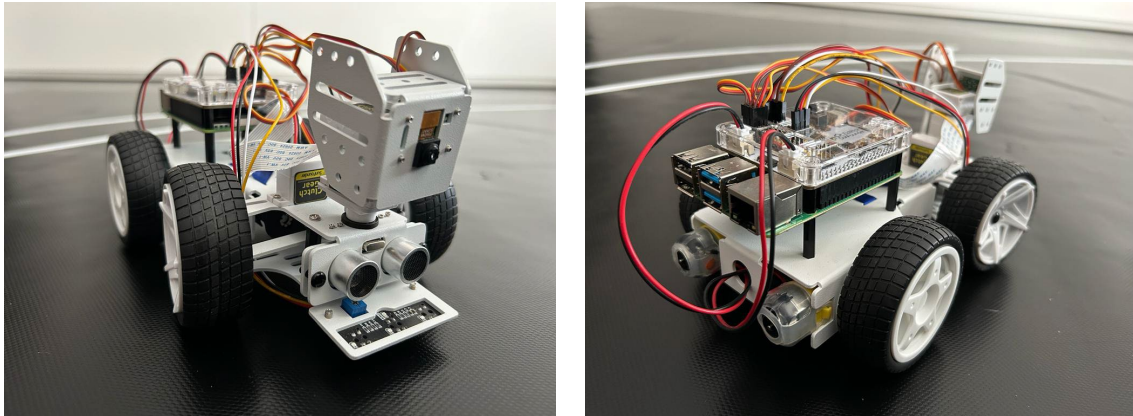
The system requires both lines to estimate the centre of the lane, as it takes the average slope and intercept between the left and right lane line. To overcome this issue, the vehicle could predict the width and the direction of the lane only by detecting the centre (left) line. This, however, would be very unreliable. This issue could also be overcome by using AIs such as YOLOP which detect the entire drivable area and the lanes.

In conclusion, although there are more complex (out of scope) solutions which can overcome these problems without the use of any other sensors (such as infrared cameras) or AI (for example YOLO Panoptic), however they rely heavily on the conditions of the environment and the road.

11. Physical implementation

11.1 Prototype details

Due to the expected difficulty of implementing the system into the eco racing car, the physical implementation of the system has been made on a small robot. Instead of developing and assembling an entire robot, which could be an entire project on its own, the robot “Picar-x” by SunFounder has been used.



Figures 76-77. Images of the Picar-x robot car (source: own content)

The robot has two motors for the back wheels, a servo used to steer the vehicle, a “Pi Camera 3” which can be controlled with 2 servos (for the tilt and pan) and other sensors which have not been used in this case. The brain of the robot consists of a Raspberry Pi 4, which is a tiny computer which runs on the Linux Operating System, serving as a more powerful and capable alternative to Arduino boards.

The raspberry has pins to control actuators (servos and motors) and read sensors (cameras and infrared sensors).

11.2 Code adaptations

In this case, the prototype’s functionality has been limited to only have the lane detection system work in real time, as the raspberry is still not powerful enough to have both systems. As for the pathfinding system, it has also not been implemented, due to the same restrictions of being able to track the position of the prototype.

As in the implementation in BeamNG, a control system has been developed in order to control the steering with the output of the lane detection system. The control system uses a Python library made by SunFounder specifically for the Picar-x robot that enables the servos and motors to be controlled with Python scripts.

Apart from creating the control system, many other parameters have been tweaked in order to adapt the lane detection system to the new position and angle of the camera, as well as its different resolution and Field of View (FOV).

11.3 Found issues

The main issues with the implementation were arisen from the limitations of the camera. Mainly, when on a turn the interior line stops being seen by the camera (due to its position and FOV) before the prototype starts to steer, not giving it enough time to react to the turn.

Despite the issue being part of the properties of the camera and not the software, it could be solved by adapting the line detection system to the situation. The change in question, was enabling the line detection system to still steer when only one of the lines was detected (a previously made suggestion, see 10.3), steering the car in the direction of the seen lane marking.



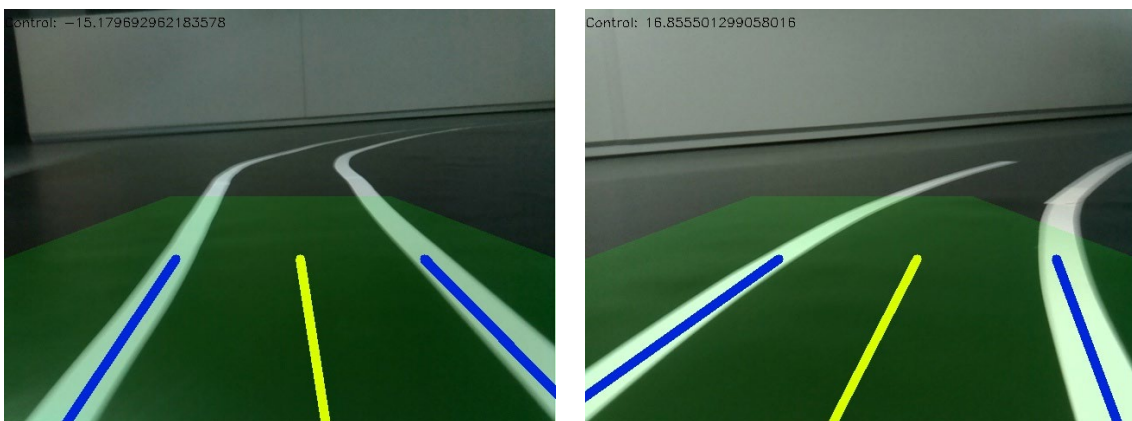
Figure 78. Capture from the vehicle's camera, showing only one of the lines visible on a turn. (source: own content)

Another issue was its sensitivity to lighting. When on a low-light environment, the prototype had trouble detecting the lines. To solve this problem, lights were used (as with a real car).

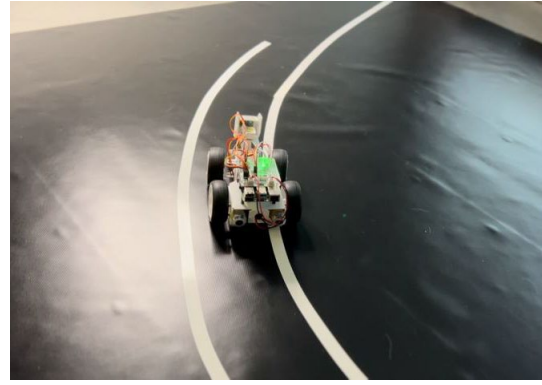
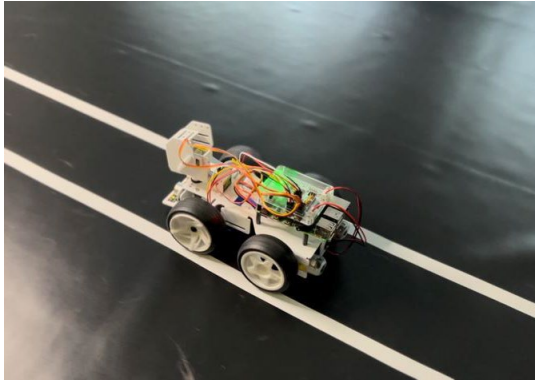
11.4 Testing the line detection system

The test track which has been used to test the vehicle consists of a black mat with white adhesive tape to represent the lanes. The lanes have been made slightly tighter than usual to help solve the issues previously mentioned with the FOV. For this test, the track had a straight line and a curve.

After applying the adaptations and fiddling with the configuration of many aspects of the system such as the PID, error weights, steering angle and the mask size and position; the prototype was able to correctly steer to keep itself within the lanes, both in straight sections and in curves.



Figures 79-80. Captures from the lane detection system in the prototype (source: own content)



Figures 81-82. Images of the prototype in action (source: own content)

Despite being able to autonomously follow the lanes without much trouble, it was a bit wobbly, constantly steering left and right, and as mentioned before, it was very sensitive to the light conditions.

In conclusion, once again, the system was successful in a real physical environment. Having said that, it does have room for improvement, though major upgrades would require a structural change of both the prototype and the systems.

12. Technical details

In this section, the more technical features (code, libraries used) of the different parts of the project will be described in more detail, as well as the link and installation steps of the code that has been developed. This has been done in order to separate the more technically dense aspects from the main explanation.

All of the scripts have been coded with Python, being one of the most used languages worldwide, especially for tasks involving computer vision and Artificial Intelligence. However, its main drawback is that it is not possible to run the code without compiling it with an interpreter, or in other words, it cannot be run as an “.exe” file without previous installation of Python and the required libraries. Its speed is also limited by the same reason.

12.1 Line Following system

The object detection system is based on a library called OpenCV (Open Computer Vision), which is an open-source library with tools for computer vision and processing for images, videos, and real-time footage. OpenCV supports a variety of programming languages such as Python, C++, or Java. It was released in June 2000. (Bradski, 2000)

OpenCV also works in combination with another library called “NumPy”, which provides functions for mathematical and statistical operations (mainly with arrays). (Harris, et al., 2020) The two libraries work in tandem by treating images like arrays (or matrices), allowing them to be handled and processed with ease. OpenCV also implements various algorithms which have been used for the lane detection system, which are: Canny Edge Detection, Gaussian Blurring and the Hough Lines Transform; and functionalities for the visualisation of output images.

Another library which has been used for the visualisation of various data and images, mainly graphs (figs 21, 22 and 73), is “Matplotlib”, which is an extension of NumPy.

12.2 Object Detection system

Like the line following system, the object detection system also makes use of NumPy and OpenCV for the handling of images and to display the results from the models. Other libraries have been required to implement deep-learning and AI functionalities. The most used ones are PyTorch (Paszke, et al., 2019) and TensorFlow (Abadi, et al., 2015). Both have their pros and cons and, in this case, PyTorch has been used.

12.2.1 Testing an object detection model

With PyTorch, to test an object detection model (get the output from an image) the model has to be downloaded and imported into the project, to then run a function to process the image.

AI models are composed of configurations (often coming from a config file) and the weights (parameters of the neural network). The models can usually be downloaded and imported into the project with the functions (`torchvision.models`) included in the PyTorch library.

```
import torchvision.models as models
resnext50_32x4d = models.resnext50_32x4d()
```

Or the models can also have their own libraries which download and load the model:

```
from torchvision.models.detection import fasterrcnn_resnet50_fpn_v2,
FasterRCNN_ResNet50_FPN_V2_Weights

weights = FasterRCNN_ResNet50_FPN_V2_Weights.DEFAULT
model = fasterrcnn_resnet50_fpn_v2(weights=weights)
```

Alternatively, the models can be manually downloaded and imported into the project, which is necessary for loading exported models that have been trained.

Once the model is loaded, the images can be processed using `model(image)`, which will output a list of the predictions/results. The results can be then displayed either by drawing the bounding boxes with the OpenCV library (mentioned earlier), or by using another function from PyTorch:

```
from torchvision.utils import draw_bounding_boxes

prediction_output = model(batch)
labels = [weights.meta["categories"][i] for i in prediction["labels"]]
box = draw_bounding_boxes(img, boxes=prediction["boxes"],
labels=labels,
width=8]
```

YOLO, however, has its own library, derived from PyTorch, although it can also be used with PyTorch directly. For the object detection system, the specific library for YOLO has been used. With the library, the model must be downloaded, and its file imported into the project:

```
from ultralytics import YOLO

model_path = "./models/yolov8-m.pt"
model = YOLO(model_path)
```

Once the model has been loaded, the images with the function `model.predict()`. This function also has the built-in option to show the results:

```
results = model.predict(input_image, device="cpu", show=True, save=True)
```

12.2.2 Training the object detection model

The steps needed to train an object detection model differ depending on whether PyTorch or YOLO is used. Those differences include the structure of the dataset that is used to train the model and the function and parameters used to start the training.

Dataset Structure

In object detection datasets, there is usually two kinds of files: the images and the labels. These are normally split up into a training (train) and an evaluation (val) group (see 8.2). With the YOLO library, datasets are required to have the following file structure:

There has to be a folder named “images” containing the images and another one named “labels” for the labels. Inside each folder there has to be a “train” folder and a “val” folder.

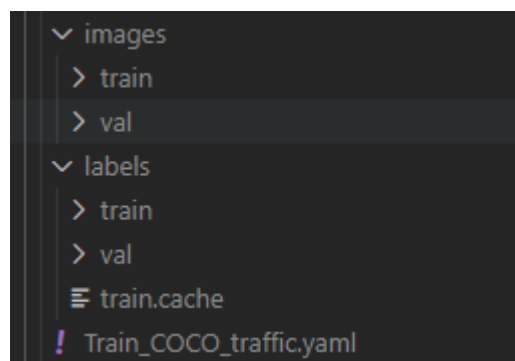


Figure 83. Screenshot showing the file structure of the dataset (own content)

Inside the root folder, there also has to be a “.yaml” file, which will contain the configuration and information about the dataset and the training process. This file must contain the path to where the images are located, both for the “train” and “val” images; the number of classes and a list of all of the classes. Other optional configurations can be added. With the essential requirements, it should look the following:

```
train: ./02_coco-traffic/images
val:   ./02_coco-traffic/images

nc: 84 #Number of classes
names: ['person', 'bicycle', 'car', ..., 'traffic_light_na']
```

Apart from the config files, the label files must also have a specific format to indicate the objects present in the image. For each object in the image the format must be the following:

```
<class_id> <x1> <y1> <x2> <y2> ... <xn> <yn>
```

With only two points (x1, y1) (x2, y2), the model will only be able to provide a bounding box to indicate the object. For semantic segmentation models, however, more points can be given, resulting in a polygon describing the shape of the object.

Training the model

With YOLO, the training can be started with the function `model.train()`. The parameters required for training are data (the “.yaml” file), epochs (the number of iterations the model will go through when training), imgsz (the size the images will be compressed to) and device (cpu or gpu).

```
model.train(data=yaml_file, epochs=30, imgsz=640, device="cpu")
```

Once the training commences, the model will start going through the images. The process will take a long time. Every epoch, the current iteration of the model will be saved. While it is training, the current statistics of the model such as precision and time per iteration will be shown in the command line:

Epoch	GPU_mem	box_loss	seg_loss	cls_loss	dfl_loss	Instances	Size			
9/10	0G	0.9911	2.249	1.345	1.4	109	640:	21%		191/902
10/10	0G	0.9472	2.212	1.289	1.373	137	640:	40%		364/902
10/10	0G	0.9396	2.202	1.286	1.367	84	640:	100%		902/902

Training a model, takes a lot of time and resources. To accelerate the process, a gpu can be used instead of using the cpu. This requires a CUDA compatible gpu. In this case, it was not possible to get a gpu with CUDA to work. More information on this can be found in the Nvidia Toolkit documentation.

12.3 Downloading and installing

The link to all of the code developed for this project has been uploaded to a GitHub repository, it can be downloaded directly from: https://github.com/Algreen333/TR_AutonomousDriving, or it can be cloned using the command with “git” installed:

```
git clone https://github.com/Algreen333/TR_AutonomousDriving.git
```

In it, there is a folder for each one of the systems developed. Each system has its own libraries and dependencies. To avoid compatibility issues a virtual environment should be created using the following command, but first make sure Python is correctly install (preferably version 3.11):

```
python3 -m venv ./PATH_TO_VENV
```

Then it should be activated using the command: `./PATH_TO_VENV/Scripts/activate`

Once the virtual environment is created and running, the libraries can be installed. To do so, there is a file named “requirements.txt” containing all of the dependencies for each system. To install the libraries from the requirements text file run: `pip install -r "requirements.txt"`

With this done, there should be no issues when running the code apart from some configuration parameters which are specified at the start of the scripts.

13. Conclusions

At the start of the project, the three main objectives proposed were the following:

- Creating an autonomous driving system
- Implementing and training the driving system in simulations
- Testing the driving system with a physical prototype

In the end, it has been able to prove that the approach of this project to making an autonomous driving system, through an exhaustive process of development and testing, is successful; having ended up with a physical prototype (though limited) and an implementation in simulation which were able to keep the vehicles within the lanes without human intervention.

However, not everything was accomplished on its entirety. Despite the lane detection system being able to react to steer the vehicle, the other two systems developed were not fully implemented into simulators and the prototype, limited by technical aspects such as hardware limitations. With more time and resources, more complex solutions could be implemented.

As for the training and simulations, no simulators were developed as it would have supposed much more effort and complexity. However, BeamNG was used in order to simulate a realistic environment and test the lane and object detection systems, having the same, if not better, results as using a specifically designed simulator.

To train the object detection model, as no simulators were developed, datasets were used, being a better option. Despite having had to use external computing power, the final object detection model still has many limitations, such as its performance. However, it accomplishes the objective of detecting cars and other obstacles.

The pathfinding system was designed in a more theoretical manner, enough to understand its usage and functionalities, but not to be implemented in simulations or the physical prototype.

In relation to the implementation in a physical prototype, despite not being able to implement the system into the Eco Racing car, the driving system was successfully implemented into the Picar-x robot prototype.

Apart from the objectives, it has to be mentioned that this project has been able to provide an in-depth view into the state-of-the-art of computer vision and AI, focusing on the aspects related to autonomous driving.

To conclude, despite the ambition of this project and its resulting limitations, it serves as a proof of concept, being able to provide an example of how a real autonomous driving system could have been developed, its difficulties and also how an AI is trained. It also shows the potential applications it may have, not just restricted to driving cars autonomously.

14. References

- Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., . . . Zheng, X. (2015). TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems.
- Ahmadi Mehr, R. &. (2022). Skin Cancer Detection Based on Deep Learning. *Journal of biomedical physics & engineering*, 12 (6), 559–568. doi:<https://doi.org/10.31661/jbpe.v0i0.2207-1517>
- Alizadeh, F., Mniestri, A., & Stevens, G. (2022). The Reverse Turing Test: Being Human (is) enough in the Age of AI. *CEUR Workshop Proceedings*.
- Anyoha, R. (2017, August 28). *The History of Artificial Intelligence*. Retrieved September 17, 2023, from Harvard University: <https://sitn.hms.harvard.edu/flash/2017/history-artificial-intelligence/>
- Boora, D. (2023, February 12). *Here's What A Level 3 Self Driving 2024 Mercedes-Benz S-Class And EQS Can Really Do*. Retrieved May 5, 2023, from HotCars: <https://www.hotcars.com/2024-mercedes-benz-s-class-eqs-level-3-self-driving-explained/>
- Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*. Retrieved from <https://opencv.org/>
- Geiger, A., Lenz, P., & Urtasun, R. (2012). Are we ready for Autonomous Driving? The KITTI Vision Benchmark Suite. *Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Goodman, D. &. (1997). Man versus machine: Kasparov versus deep blue. *ICGA Journal*, 20(3), 186-187. Retrieved from https://web.archive.org/web/20181030051029id_/https://content.iospress.com/download/icga-journal/icg20-3-08?id=icga-journal%2Ficg20-3-08
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., . . . Oliphant, T. (2020, September). Array programming with NumPy. *Nature*, 585(7825), 357--362. doi:10.1038/s41586-020-2649-2
- Houben, S., Stallkamp, J., Salmen, J., Schlipsing, M., & Igel, C. (2013, November 5). Detection of Traffic Signs in Real-World Images: The German Traffic Sign Detection Benchmark. Retrieved September 26, 2023, from https://benchmark.ini.rub.de/gtsdb_news.html
- How Autonomous Vehicles Work*. (2021, June 12). Retrieved April 9, 2023, from Let's Talk Autonomous Driving: <https://ltad.com/about/how-autonomous-vehicles-work.html>
- Johnson, S. (2022, March). *The Turing test: AI still hasn't passed the "imitation game"*. Retrieved September 16, 2023, from Big Think: <https://bigthink.com/the-future/turing-test-imitation-game/>
- Joseph Redmon, S. D. (2016, May 9). You Only Look Once: Unified, Real-Time Object Detection. *arXiv*. Retrieved from <https://arxiv.org/abs/1506.02640v5>

- Kobylnski, K. (2019, November 28). *What is adaptive cruise control and how does it work?* Retrieved May 5, 2023, from Tomtom: <https://www.tomtom.com/newsroom/explainers-and-insights/what-is-adaptive-cruise-control/>
- Koltun, A. D. (2017). CARLA: An Open Urban Driving Simulator. 1-16. Retrieved from Carla.org: <http://proceedings.mlr.press/v78/dosovitskiy17a/dosovitskiy17a.pdf>
- Laukkonen, J. (2021, 25 July). *How Does Lane Centering Work?* Retrieved May 3, 2023, from Lifewire: <https://www.lifewire.com/how-does-lane-centering-work-5188779>
- Lin, T.-Y., Maire, M., Belongie, S., Bourdev, L., Girshick, R., Hays, J., . . . Dollár, P. (2014, May 1). Microsoft COCO: Common Objects in Context. doi:<https://doi.org/10.48550/arXiv.1405.0312>
- McCarthy, J. (2007, November 12). *WHAT IS ARTIFICIAL INTELLIGENCE?* Retrieved September 17, 2023, from John McCarthy's Home Page: <http://www-formal.stanford.edu/jmc/>
- Neuhold, G., Ollmann, T., Rota Bulò, S., & Kotschieder, P. (2017). The Mapillary Vistas Dataset for Semantic Understanding of Street Scenes. In *International Conference on Computer Vision (ICCV)*.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., . . . Chintala, S. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. Retrieved from <https://doi.org/10.48550/arXiv.1912.01703>
- Prasad, K., & Kumari, M. (2020). *A review on mathematical strength and analysis of Enigma*. Department of Mathematics, Central University of Jharkhand, India, 835205. Retrieved from <https://arxiv.org/pdf/2004.09982.pdf>
- Ren, S., He, K., Girshick, R., & Sun, J. (2015, June 4). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. doi:<https://doi.org/10.48550/arXiv.1506.01497>
- Russell, S. J., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Prentice Hall.
- Tesla. (2023, January). *Tesla Vehicle Safety Report*. Retrieved April 9, 2023, from Tesla: <https://www.tesla.com/VehicleSafetyReport>
- The 6 Levels of Autonomy Explained*. (2022). Retrieved April 4, 2023, from Synopsis: <https://www.synopsys.com/automotive/autonomous-driving-levels.html>
- Turing, A. M. (1950). Computing Machinery and Intelligence. In A. M. Turing, *Mind, New Series*, Vol. 59, No. 236 (Oct., 1950) (pp. 433-460). Oxford University Press on behalf of the Mind Association.
- Waymo. (2023, February 28). *Safety Performance of the Waymo Rider-Only Automated Driving System at One Million Miles*. Retrieved May 10, 2023, from Waymo: <https://storage.googleapis.com/waymo-uploads/files/documents/safety/Safety%20Performance%20of%20Waymo%20RO%20at%201M%20miles.pdf>

- Waymo. (n.d.). *Company - Waymo*. Retrieved May 5, 2023, from Waymo: <https://waymo.com/intl/es/company/#story>
- Weizenbaum, J. (1966). ELIZA—a computer program for the study of natural language communication between man and machine. *Communications of the ACM*, 9.1, 36-45. doi:10.1145/365153.365168
- Wu, D., Liao, M.-W., Zhang, W.-T., Wang, X.-G., Bai, X., Cheng, W.-Q., & Liu, W.-Y. (2022). Yolop: You only look once for panoptic driving perception. *Machine Intelligence Research*, 1-13.